

Classe

Fondamenti di Programmazione

26/09/2023 – Lezione Introduttiva

Versione di python ≥ 3.9 , ipython (interprete-ipython) e l'uso di Spyder

Se si vuole si possono fare gli esercizi al termine di ogni capitolo sul libro "Pensare in Python" scaricato in pdf.

Molto consigliata la documentazione ufficiale di Python ma è solo in inglese.

Su q2a conosceremo circa 3 tutor per il nostro canale che ci aiuteranno con installazioni e preparazione all'esame.

Contatti: Nel file inviato dal prof o nella foto sul telefono

Per urgenze conviene inserire nell'oggetto della mail [FdP24]

Modalità di Esame

Durata: 2.5/3h

2/3 di voto derivano dai 7 esercizi dell'esame (4 facili, 1 difficile sulle immagini, 2 ricorsivi)

1/3 del voto di esame deriva dagli HomeWork (HW) e si hanno 15 giorni per realizzarli. Nel caso in cui te li dimentichi o andassero male vi sono degli HW di recupero senza scadenza.

Servono 4 HW superati per accedere all'esame

Valutazione

Agli hw e all'esame riceviamo il testo del problema, dati di input e dati di output atteso. (Praticamente dobbiamo scrivere il codice che dall'input ci possa portare agli output).

Se il nostro output (x) sarà uguale all'output atteso (expected_X) allora avremo superato il test.

Il file che dovremo eseguire il giorno dell'esame sarà grade.py, cioè il main.

Cosa Vedremo

Vedremo soprattutto il linguaggio di alto livello, che risulta più semplice. Il linguaggio di basso livello è più complesso e lo vedremo molto probabilmente in architettura degli elaboratori nel 2 semestre

Comandi Utili

Se si vuole castare un valore per renderlo integer si può inserire **int(..)** davanti l'operazione/variabile.

Print è il comando base per stampare delle informazioni. `>>>print ("ciao")`. Di base il comando print manda a capo una volta che ha stampato. Per evitare che questo accada ed inserire uno spazio tra più elementi si può inserire `... = "\t"` (in cui t sta per tabulazioni). Oppure `"\n..."` per scrivere qualcosa una volta andati a capo.

tab sulla tastiera completerà l'istruzione che stavo scrivendo.

help(...) → Se non ci ricordassimo il funzionamento di una funzione possiamo chiedere aiuto con la funzione

Per incrementare in modo costante una variabile possiamo utilizzare il metodo classico `x = x + 1` o scriverlo come `x += 1` (e la stessa cosa vale con gli altri operatori `x *= 2`, `-=`, etc.)

str.lower(...) rende tutti i caratteri di una stringa minuscoli

str.find() ci restituisce l'indice in cui si trova quella lettera nella stringa. Molto molto simile è **str.index()** che ci dice in quale indice inizia la parola inserita tra virgolette.

str.replace(valore da togliere , valore nuovo) sostituisce un nuovo valore ad un precedente già contenuto in una stringa/lista/tupla. Tuttavia con tipi Immutabili (come stringhe e tuple) l'operazione avviene IN PLACE e non c'è valore di ritorno e quindi il comando andrà perso. Per salvare il risultato della funzione bisognerà inserirlo dentro una nuova variabile.

str.split() si applica su una stringa e rende una lista. Split rimuove un determinato elemento dalla stringa e separa tutto ciò che c'è prima da ciò che c'è dopo, rendendoli come due elementi diversi in una lista.

```
stringa = '3,0,4,0,3,1,0,1,0,1,0,0,5,0,4,2'

L = stringa.split(',') # indichiamo a split su che sottostringa "splittare" ossia
                       # come suddividere la stringa in elementi della lista
```

str.join() è l'esatto opposto di split e unisce più elementi (necessariamente di tipo str) di una lista in una sola stringa, separati dalla stringa indicata prima del punto.

	str.isnumeric()	str.isdecimal()	str.isalnum	str.isalpha()
	<code>isnumeric()</code>	<code>isdecimal()</code>	<code>isalnum()</code>	<code>isalpha()</code>
abcdef123	False	False	True	False
??<ABCD!@#%&^*	False	False	False	False
abcWWWXX	False	False	True	True
01010002	True	True	True	False

Queste funzioni controllano la tipologia di stringa che stiamo analizzando, se contiene solo numeri, se contengono solo numeri decimali, solo lettere o se sono alfanumeriche etc.

Con il termine **cd** (change directory) ci possiamo spostare nelle varie cartelle del computer. Utile soprattutto nel terminale per posizionarci una cartella specifica

input() permette di stampare nel prompt una domanda a cui l'utente può rispondere.

```
n = input("scegli un numero \n")
print("il numero che hai scelto è:", n)
```



```
scegli un numero
11
il numero che hai scelto è: 11
```

%reset -f per ripulire tutto ciò che è scritto nell'interprete

Esistono anche alcune funzioni di base codificate in python, per svolgere operazioni come **sum()**, **max()** o **min()**, **count()**.

Ordine degli Operatori

Per ricordare l'ordine di importanza degli operatori si può ricordare con l'acronimo **PEMDAS**:

- Parentesi
- Elevamento a Potenza (4^{**2})
- Moltiplicazioni (*)
- Divisioni (/) o (// per divisioni intere) (per sapere quale è il resto in una divisione intera si usa %) ($5\%2=1$)
- Addizioni (+)
- Sottrazioni (-)

Per usare la radice quadrata bisogna importare in cima al codice la libreria mat (**from math import ***) e utilizzare il comando **sqrt(n)** o elevare tale numero a 0.5 ($n^{**0.5}$)

27/09/2023

Il Prompt o Terminale è una finestra nella quale inseriamo le istruzioni che verranno eseguite dal terminale stesso.

Per eseguire i programmi apriamo il terminale, gli diciamo il programma da lanciare (nel nostro caso python) e di seguito il file.py che deve aprire

Per programmare sarà necessario utilizzare degli editor di testo.

L'area a sinistra è dove andremo a scrivere il codice del programma.

L'area in basso destra si divide in Python Console, cioè il prompt/terminale in cui verranno svolte le operazioni dettate dal codice del programma (funzione >>>runfile), e la sezione History che funziona da cronologia per le operazioni svolte.

L'area in alto a destra contiene la Variable Explorer che fornisce informazioni aggiuntive sulle stesse. La prima colonna ci mostra il nome delle variabili, nella seconda il tipo di variabili, nella terza i byte utilizzati da quella variabile (1byte per ogni carattere utilizzato) e nella quarta colonna il valore della variabile.

Il consiglio grande è di programmare un pezzo di codice alla volta, avviando i, programma più volte nella scrittura del codice, per rendere più semplice l'individuazione degli errori.

28/09/2023

Pwd → Print Working Directory per farci dire il percorso in cui ci troviamo

Ctrl1 → Attivi o Disattivi i commenti

Il pallino rosso si chiama Break Point, o punto di fermata, e blocca lo status delle variabili.

Per attivare il Debug è necessario inserire un BreakPoint ed è importante disattivare la mod di debug prima di avviare nuovamente il programma. Il debug risulta molto utile soprattutto per monitorare l'evoluzione delle variabili mano a mano che svolgiamo le operazioni.

Il numero massimo di bit che posso codificare con Python dipende dalla memoria del mio computer e lo otteniamo facendo $2^n - 1$. Se i bit superano il numero massimo si va in overflow.

Esistono operazioni matematiche e operazioni booleane, i cui risultati possono essere solo di due tipo (true/false)

Operatori di Confronto

Ci permette di confrontare due variabili. Interrogano il computer e restituiscono un risultato Boolean, che può essere solo di due tipi (es. True/False)

$X == y$	$X != y$	$X > y$	$X < y$
X è uguale a y?	X è diverso da y?	X è maggiore/minore y	

Is non è propriamente un operatore di confronto. Esso controlla se le due variabili sono salvate nello stesso punto in memoria. Si differenzia da == che invece confronta il contenuto delle variabili

Tipi di Variabile

Immutabili

Integer - int (Numeri Interi)

Floating Points - float (numeri razionali e irrazionali con la virgola)

String - str (testi, segnalati utilizzando gli apici ' ' o le virgolette " ")

(Per stampare anche gli apici o le virgolette bisogna inserire lo \ (usato per fare escape) davanti agli apici o le virgolette in questione.)

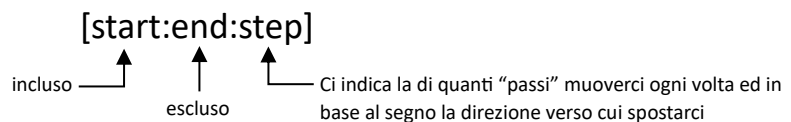
Le string possono anche essere sommate, per stamparle in un'unica frase, o moltiplicate per essere stampate più volte → `print(stringa*2)`.

In alcuni testi potrebbero trovarsi degli \n o \t che DEVONO essere stampati, e non svolgere i comandi. Possiamo quindi mettere una r (raw string) prima dell'inizio della stringa, oppure inserire uno \ prima di \n o \t per non farle funzionare)

Le stringhe vengono codificate come vettori e con il comando `len(...)` possiamo vedere la lunghezza, cioè il n. di caratteri, di quella stringa. Per sapere quale carattere si trova in un determinato punto della stringa si possono usare le [] inserendo tra le quali l'indice del vettore, quindi la posizione di cui vogliamo sapere a lettera.

car.	i	a	c	o	p	o		m	a	s	i
idx	0	1	2	3	4	5	6	7	8	9	10

Se vogliamo estrarre una parte della string possiamo effettuare uno slicing semplicemente fornendo l'intervallo che vogliamo stampare [2;6]. Tuttavia questo comando include il punto di inizio ma NON il punto di arrivo. Quindi ciò che è contenuto nell'indice 6 non verrà stampato. È possibile anche escludere volontariamente lo start e l'end. Con [1:] stamperemo dal 1 carattere in poi 1 escluso. Con [:6] stamperemo dall'inizio fino alla 6° posizione esclusa.



Con questo comando ci si può anche "muovere al contrario" facendo partire l'intervallo da -1, che sarebbe l'ultimo carattere → [-1:-6:-1] → il terzo indice rappresenta lo step e ci dice di quanti "passi" muoverci ogni volta ed in base al segno ci sposteremo a dx (segno +) o verso sx (segno -). In questo caso con step = -1 ci sposteremo di 1 posizione alla volta da destra verso sinistra. Se start >= end avremo in out una stringa vuota.

+	-	+	-	+	-	+	-	+	-	+	-	+
	P		y		t		h		o		n	
+	-	+	-	+	-	+	-	+	-	+	-	+
0	1	2	3	4	5	6						
-6	-5	-4	-3	-2	-1							

Supponiamo però di non sapere il punto in cui inizia la parola che stiamo cercando nella stringa. Esiste dunque il comando **nome.index(...)** che ci dice in quale indice (il punto preciso della stringa) inizia la parola inserita tra virgolette.

```
start = nome.index('copo')
start
2
```

Le String di base vengono codificate come immutabili da python. Tuttavia esistono degli escamotage che ci permettono di modificarli. Come **str.lower(...)** che rende tutti i caratteri di una stringa minuscoli

Boolean - bool (può essere solo di due tipi, per esempio True/False)

(Tuttavia nella creazione delle variabili Bool a True è stato attribuito il valore 1 e a False il valore 0. Sono quindi valori booleani interi → True + True = 2)

NoneType – None (indica il concetto di niente)

Tuple – tuple (è una sequenza di valori separati da virgole)

I valori all'interno della tupla possono essere di qualsiasi tipo e sono indicizzati tramite numeri interi (un concetto molto simile a quello dei vettori in VB). Così come le stringhe anche le Tuple supportano operatori di index e slicing, ma allo stesso tempo anche gli operatori di confronto. Tuttavia, per gli operatori di confronto, Python inizia a confrontare il primo elemento di ciascuna sequenza. Se sono uguali, passa all'elemento successivo e così via finché non trova due elementi diversi; (qui viene presa una decisione).

Le Tuple sono MOLTO utili soprattutto per avere più valori di ritorno da una funzione.

```
def div_mod(a,b):  
    '''my function to compute integer  
    division. Return quotient and mod'''  
    q = a // b  
    r = a % b  
    return q, r # e' una tupla che contiene quoziente e resto
```

Mutabili

Liste – list

Essendo mutabili possiamo effettuare l'item assignment per modificare i singoli elementi contenuti nella lista (cosa che non potevamo fare con le Tuple). `Lista[1:2] = [12]` → andremo ad inserire il valore 12 al posto dei valori precedentemente contenuti nell'intervallo [1:2].

Per cercare all'interno di TUTTA la lista posso usare il comando `in`. Può risultare utile anche per capire se un elemento si trova o meno all'interno di una lista. Per sapere l'indice (cioè il punto preciso della lista in cui si trova un determinato valore) posso usare il comando **lista.index()**

Con il comando **lista.append(i)** possiamo aggiungere un nuovo elemento in fondo alla lista. Tuttavia questo comando modifica la lista "in-place", cioè senza valori di ritorno.

Un altro comando per inserire elementi in coda è **lista.extend()**, che torna molto utile per unire due liste.

Per rimuovere elementi da una lista possiamo utilizzare due funzioni. **lista.remove()** ("il contenuto che vogliamo rimuovere") oppure **lista.pop()** (indice del valore da cancellare).

Operazioni su String, Tuple, List

Operation		String s = 'hello' w = 'l'	Tuple s = (1,2,3,4) w = (5,6)	List s = [1,2,3,4] w = [5,6]
Length	<code>len(s)</code>	5	4	4
Select	<code>s[0]</code>	'h'	1	1
Slice	<code>s[1:4]</code>	'ell'	(2, 3, 4)	[2, 3, 4]
	<code>s[1:]</code>	'ello'	(2, 3, 4)	[2, 3, 4]
Count	<code>s.count('e')</code>	1	0	0
	<code>s.count(4)</code>	error	1	1
Index	<code>s.index('e')</code>	1	--	--
	<code>s.index(3)</code>	--	2	2
Membership	<code>'h' in s</code>	True	False	False
Concatenation	<code>s + w</code>	'hello!'	(1, 2, 3, 4, 5, 6)	[1, 2, 3, 4, 5, 6]
Minimum Value	<code>min(s)</code>	'e'	1	1
Maximum Value	<code>max(s)</code>	'o'	4	4
Sum	<code>sum(s)</code>	error	10	10

Insiemi - set

Un set è una collezione di elementi SENZA ORDINE, SENZA DUPLICATI (gli elementi uguali vengono cancellati automaticamente) e MUTABILI, ma possono contenere solo elementi IMMUTABILI (come stringhe, interi, tuple). Una cosa importante dei set è che essi NON sono ordinati e dunque non sono nemmeno indicizzati. Per accedere i set dobbiamo necessariamente utilizzare i cicli.

Per la creazione di set si utilizzano le parentesi graffe a_set = {} (fare attenzioni se vogliamo dichiarare un set vuoti perché con questo metodo si rischia sia considerato come dictionary che vedremo dopo), oppure con set(valori). I set si possono creare anche dando in ingresso una sequenza di stringhe, liste, tuple etc. che chiameremo container, a patto che gli elementi del container siano tutti IMMUTABILI. Ogni elemento della sequenza diventerà un elemento del set.

```
set(sequenza) #set(container) pero ciascun elemento del container deve avere
elementi immutabili

my_set = set([1, 2, 3, 4])
print(type(my_set))
my_set

<class 'set'>
{1, 2, 3, 4}
```

I sets sono particolarmente utili per testare l'appartenenza di un elemento ad un insieme ed eliminare eventuali elementi duplicati. Per esempio se vogliamo sapere le singole lettere contenute nella stringa="bananarama" basterà trasformarla in set (dandogli in pasto la stringa stessa → set("bananarama") e stamparla. Infatti, essendo che i set non possono avere elementi duplicati, le lettere che si ripetono vengono eliminate automaticamente al momento della conversione. Tuttavia si perde l'ordinamento della stringa.

```
set('bananarama')
#NB: si perde ordinamento stringa

{'a', 'b', 'm', 'n', 'r'}
```

(stessa identica cosa si potrebbe fare dando in pasto una lista di stringhe. In questo caso però eliminerebbe le stringhe doppie, e non le lettere)

```
s = set(['apple', 'orange', 'apple',
        'pear', 'orange', 'banana'])
print(s) # i doppiati sono scomparsi

{'orange', 'pear', 'banana', 'apple'}
```

Un altro esercizio semplice con i set sarebbe quello per controllare se una stringa abbia caratteri duplicati o meno. In questo caso basterà convertire la stringa in set (così da eliminare i doppiati) e verificare se la stringa e il set hanno lunghezza uguale.

```
stringa = 'pippo'
f'{stringa} contiene duplicati' if len(set(stringa)) != len(stringa) else f'{stringa} NO'
'pippo contiene duplicati'
```

I Set sono molto comodi per:

- **Rimuovere duplicati/Calcolare valori unici.**
- **Modellare relazioni insiemistiche** (se vi chiedono unione meno intersezione fra sequenze) (trova gli elementi di A che non sono in B ma sono anche in C)
- **Testare se qualcosa è contenuta in una struttura dati** (e nel farlo sono molto più veloci dell'operatore in e delle Liste)

Manipolazione dei set

Aggiungere →

```
s.add(el) # Oppure: s |= {el}
s.update(altra_set) # Oppure: s |= <altra_set>
```

Rimuovere →

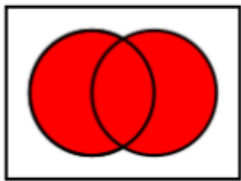
```
el = s.pop() # solleva KeyError se vuoto
s.remove(el) # solleva KevError se non contiene el
s.discard(el) # non solleva nessun errore
```

Operazioni sui set

$s = \{1, 4, 6, 7\}$

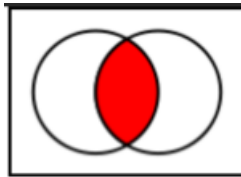
$t = \{1, 2, 3, 4, 7, 9\}$

Unione (or)



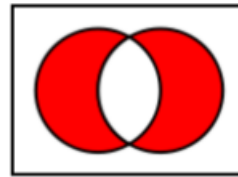
```
s | t
{1, 2, 3, 4, 6, 7, 9}
```

Intersezione (and)



```
# intersezione
s & t # equivalente s.intersection(t)
{1, 4, 7}
```

(Differenza simmetrica) - (xor)



```
# elementi di S che non sono in T unito
# agli elem di T non in S
print(s.symmetric_difference(t))
s ^ t # forma concisa
{2, 3, 6, 9}
{2, 3, 6, 9}
```

```
# Unione delle differenze
print(s - t | t - s)
# Unione meno intersezione
(s | t) - (s & t)
{9, 2, 3, 6}
{2, 3, 6, 9}
```

Dizionari – dict ← (Li useremo per Istogrammi, Frequenze, Conteggi)

nome_dizionario = { <chiave1> : <valore>, <chiave2> : <valore> }

I dizionari sono una raccolta di elementi chiave-valore. Sono simili alle liste ma raccolgono simultaneamente indici (chiamati chiavi) e valori (associati alla chiave di riferimento). Ciascuna chiave è associata ad un solo valore, e per questo si dicono univoche, e l'associazione tra chiave e valore viene detta coppia chiave-valore e costituisce un elemento del dizionario.

Dunque, per accedere e manipolare il dizionario, abbiamo bisogno delle chiavi (key) che funzionano da indici. Se la chiave non è presente otterremo un `KeyError`. Possiamo controllare se la chiave è nel dizionario con l'operatore di appartenenza `in`.

Per accedere agli elementi del dizionario possiamo utilizzare diversi comandi:

- `dict[key]` → Accede valore associato a quella chiave
- `dict.get(key)` → Accede valore associato a quella chiave
- `dict.keys()` → Iterare solo sulle chiavi. Restituisce una lista con dentro tutte le chiavi.
- `dict.values()` → Iterare solo sui valori. Restituisce una lista con dentro tutti i valori.

- **dict.items()** → Iterare su coppie (chiave, valore). Restituisce una lista di tuple che contengono chiave-valore.

Per modificare o aggiungere valori:

- **dict[key] = value**
- **dict.update({key : value})**

Per rimuovere:

- **dict.pop(key)**
- **dict.popitem()** → Rimuove l'ultima coppia key-value
- **del dict[key]** → Per eliminare tutto il dict (senza inserire la key) o un singolo elemento

come unire due dict

Fare attenzione quando uniamo due dizionari perché se due coppie hanno chiave uguale il primo valore viene sovrascritto dal secondo.

Dimostrare come abbreviare il conteggio con il dict

I Dizionari risultano molto potenti per effettuare **caching**, per creare istogrammi o applicazioni dove devo contare frequenze/occorrenze e, in alcuni casi, possono sostituire if e elif molto densi.

03/10/2023

Cosa vuol dire Programmare?

1. Dato un problema
2. Si analizza il problema e si capisce cosa può costituire una soluzione (e cosa no)
3. Si progetta una serie di passi che risolvono il problema (algoritmo)
4. Si traducono i passi in istruzioni per il calcolatore seguendo la sintassi Python
5. Si valuta le implementazioni sui casi di test per capire se le istruzioni risolvono il problema.

Metodo Bottom Up o Top Down (file: lezione 6)

Funzioni

Esercizio – Creare la seguente tabella:

	hed		hed		hed		hed		hed
	val		val		val		val		val
	val		val		val		val		val
	val		val		val		val		val
	val		val		val		val		val
	val		val		val		val		val
	val		val		val		val		val
	val		val		val		val		val
	val		val		val		val		val
	val		val		val		val		val

Come lo abbiamo risolto:

```
head = ("| hed " * 8)
print(head)

subhead = " ---- " * 8
print(subhead)

body = (("| val " * 8) + "\n") * 5
print(body)
```

Il prototipo iniziale era una cosa del genere. Ma era subito evidente che si potesse migliorare, a partire da *8 e *5 che rappresentano il numero delle righe e delle colonne.

```
r = 5
c = 8

head = ("| hed " * c) + "\n"
subhead = (" ---- " * c) + "\n"
body = (("| val " * c) + "\n") * r
table= head+subhead+body
print(table)
```

Successivamente abbiamo quindi introdotto le variabili r e c per la conta di righe e colonne. Abbiamo poi sommato head, subhead e body dentro un'unica variabile, così da effettuare una sola stampa anziché 3. Questo passaggio ha comportato l'aggiunta di "\n" al termine di head e subhead, per evitare che venissero stampate una affianco all'altra.

```
#DEFINIZIONE DELLA FUNZIONE:
def create_table(r,c):
    #CORPO DELLA FUNZIONE
    head = ("| hed " * c) + "\n"
    subhead = (" ---- " * c) + "\n"
    body = (("| val " * c) + "\n") * r
    table= head+subhead+body
    print(table)

#INVOCAZIONE DELLA FUNZIONE
create_table(5,8)
```

A questo punto possiamo definire il tutto come una funzione, ed invocarla ogni volta che ci serve. La funzione è composta di 3 parti:

- **DEFINIZIONE:** diamo un nome alla funzione
- **CORPO:** dove ci saranno scritti tutti i passaggi da svolgere. Questi saranno individuati dal computer mettendo uno spazio con "tab" all'inizio di ogni istruzione parte del corpo della funzione.
- **INVOCAZIONE:** Da ora in poi basterà scrivere il nome della funzione perché questa venga svolta. In questo caso abbiamo rimosso le variabili r e c ed inserite direttamente come indici della funzione. Di conseguenza ogni volta che la invochiamo dobbiamo ad a dichiararne il valore.

```
#DEFINIZIONE DELLA FUNZIONE:
def create_table(r,c):
    #CORPO DELLA FUNZIONE
    head = ("| hed " * c) + "\n"
    subhead = (" ---- " * c) + "\n"
    body = (("| val " * c) + "\n") * r
    table= head+subhead+body
    return(table)

#INVOCAZIONE DELLA FUNZIONE
T = create_table(5,8)
print(T)
```

Ora l'unico miglioramento da fare è salvare il risultato della funzione. Con il print avremmo semplicemente stampato a video il risultato ma poi sarebbe finita lì. Con il return invece andiamo ad inserire il risultato all'interno di una variabile (che abbiamo chiamato T) e che potremo usare come qualsiasi altra variabile. Per fare un esempio è come se:

SENZA return

In: 10+5+1

Out: 16

CON return

In: T = 10+5+1

Out: T = 16

Le funzioni con return sono anche dette funzioni produttive, quando restituiscono valori al chiamante. Esistono infatti anche funzioni che non comprendono il return e che sono definite NON produttive, perché non restituiscono alcun valore.

05/10/2023

Moduli

Il Modulo è un file python in cui sono inserite più funzioni. Con il comando **import()** importiamo l'intero modulo e tutte le funzioni in esso contenuto. Successivamente con il comando **dir()** possiamo controllare cosa c'è dentro al modulo. Possiamo anche importare la singola funzione con il comando **from modulo import funzione**

Dopo averla importata, per avviare la funzione, sarà necessario specificare il modulo da cui arriva ed il nome della funzione → **modulo.funzione** → `lezionetable.create_table`

Sotto alla definizione di una funzione si può aprire una docstring digitando 3 apici: `'''`. In questa docstring andremo ad inserire la documentazione della nostra funzione e coinciderà con i dati stampati da python nel caso in cui si richieda `help`(su quella funzione).

Se non trova il modulo è probabile che non si trovi nel sys.path selezionato. Possiamo quindi usare il comando **wdir**: ed incollare di seguito il percorso che ci serve.

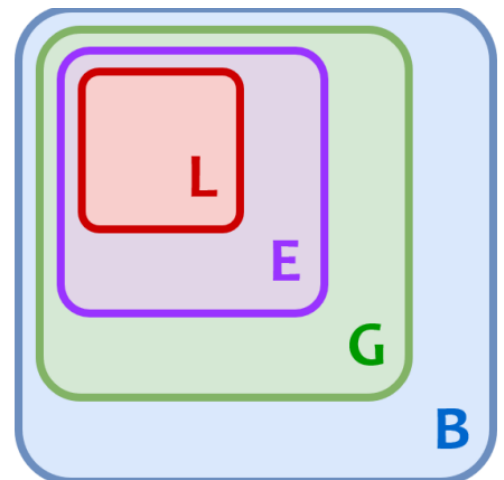
Dentro ogni modulo possiamo accedere a delle variabili particolari:

- `__file__` tiene al suo interno il path (percorso) del file corrente nel filesystem dal punto in cui viene invocato in Spyder il path e' il suo path assoluto almeno con `runfile()`
- `__name__` tiene al suo interno: il nome del modulo (che coincide con il nome del file) se usato con `import` la stringa `"__main__"` se il modulo e' eseguito come uno script.

Scope

Lo Scope è la porzione di codice in cui una variabile è disponibile. Esso può essere Locale o Globale. Considerando la variabile x, ordine è:

1. **Locale**: Se siamo dentro una funzione, l'interprete prima cerca nello scope più interno, quindi dentro la funzione.
2. **Enclosing**: Se x non è nello scope locale ma la funzione corrente è annidata in un'altra funzione, allora si cerca dentro lo scope della funzione "sopra".
3. **Global**: Altrimenti cerca la variabile nello scope globale
4. **Built-in**: Se ancora non trova niente prova nello scope delle funzioni built-in di python.
5. **Solleva un errore di tipo NameError** perché non sa risolvere il nome associato alla variabile



Funzioni Ricorsive

Una funzione è ricorsiva quando è possibile usare la stessa funzione più volte al suo interno. Un esempio potrebbe essere il fattoriale:

Assumiamo ora di non sapere come si calcola ma sapere come si calcola il fattoriale di 5. Allora sicuramente $5! = 4! * 5 \rightarrow$ Allo stesso modo $4! = 3! * 4 \rightarrow 3! = 2! * 3$ e così via...

$$n! = (n - 1)! \cdot n \\ 1! = 1$$

$$fact(n) = fact(n - 1) * n \\ fact(1) = 1$$

```
def fact(n):
    if n == 1:
        return 1
    return fact(n-1) * n

print(fact(5))
```



```
==> chiamo fact(5)
====> chiamo fact(4)
=====> chiamo fact(3)
=====> chiamo fact(2)
=====> chiamo fact(1)
=====> blocco la ricorsione con (1)
====> rit. da fact(2), fact_n 2
==> rit. da fact(3), fact_n 6
=> rit. da fact(4), fact_n 24
=> rit. da fact(5), fact_n 120
```

05/10/2023

Sintassi

```
if <condizione_booleana>:
    corpo di istruzioni se la condizione è VERA
else:
    corpo di istruzioni se la condizione è FALSE
```

È fortemente sconsigliato concatenare vari if (cioè inserire un if come corpo di un altro if) perché rende il codice lungo e difficile da interpretare. Per risolvere questo problema possiamo inserire degli “elif”.

Sintassi

```
if <condizione_booleana_1>:
    corpo di istruzioni se la condizione 1 è VERA
elif <condizione_booleana_2>:
    corpo di istruzioni se la condizione 2 è VERA
elif <condizione_booleana_3>:
    corpo di istruzioni se la condizione 3 è VERA
else:
    se TUTTE le condizioni sopra sono FALSE
```

Oppure possiamo utilizzare gli operatori booleani come not, and, or per inserire più condizioni nel corpo dello stesso if. La precedenza degli operatori booleani è la seguente: NOT, AND, OR

Operatori di Appartenenza

Un altro operatore molto utile per esprimere condizioni è l'operatore in che permette di testare l'appartenenza di un valore a una sequenza (stringhe, tuple, liste) :

Con la scrittura: >>>'pizza' in pasto>>> interroghiamo il programma e restituisce True o False a seconda che pizza appartenga o meno alla stringa, liste o tupla. Con questi operatori di appartenenza possiamo anche fare vari if.

```
In [8]: def valuta_pasto(pasto):
        if 'birra' in pasto and 'pizza' in pasto and 'ananas' not in pasto:
            print('ottimo pasto!')
        elif 'ananas' in pasto and 'pizza' in pasto:
            print('mica avrai messo l\' ananas sulla pizza? :(')
        else:
            print('potevi fare meglio...')

        pasto = 'oggi ho mangiato una pizza e ho bevuto della birra' #vino, poi ananas
        valuta_pasto(pasto)
```

12/10/2023

Ripetere le istruzioni (iterare)

Spesso i computer sono usati per automatizzare dei compiti ripetitivi. E questo concetto, nella programmazione, viene chiamata iterazione.

Ciclo For

Il Ciclo For (con significato di "Per ogni") si usa quando conoscete a priori quanti elementi dovete scandire o sapete il numero di iterazioni che si vogliono effettuare. Possiamo anche utilizzare il generatore **in range()** indicando tra parentesi il numero da cui deve partire ed il numero a cui deve arrivare. La sintassi di **in range()** è identica a quella dello slicing:

- se $start < end \rightarrow$ da start voglio andare ad end
- se $start > end \rightarrow$ da start vado a RITROSO a end

Nel processo di iterazione con **for** ricorre molto spesso di voler accedere ad un elemento ad ogni della lista e di voler sapere il corrispettivo indice di volta in volta. Per fare ciò possiamo utilizzare il comando **enumerate** in sostituzione di **in range**.

```
lista = ["a", "b", "c", "d"]  
for i, element in enumerate(lista):  
    print(i, ")", element)
```

→

0	)	a
1	)	b
2	)	c
3	)	d

Il ciclo **for** può essere terminato prematuramente con l'istruzione **break**. Viene utilizzato soprattutto all'interno degli **if**, per cui se la condizione è vera/falsa non abbiamo bisogno di continuare ad iterare e ci conviene fermarci. Spesso le istruzioni con **break** possono essere sostituite con un ciclo **While**.

Un'istruzione simile al **break** è **continue** che permette di saltare l'iterazione corrente e dunque NON svolgere il corpo del **for** per quell'iterazione.

Ciclo While

Il Ciclo **While** (con significato di "Fintanto che") si usa quando NON conosciamo a priori quanti elementi dobbiamo scandire. Il corpo del ciclo **while** deve fare sì che la condizione testata diventi **False** prima o poi durante l'iterazione, altrimenti entrere in un ciclo infinito.

```
while <condizione_booleana>: # fintantoché la condizione è VERA  
                             # allora ITERA  
    ## corpo del while  
    ## il corpo deve fare sì  
    ## che la condizione diventi FALSA  
    ## prima o poi
```

Esercizio (al 99 % ci sarà questo esercizio all'esame)

Soluzione 1

Faccio un ciclo **for**. Tuttavia mi accorgerò che non elimina tutte le stringhe con **th** all'interno. Questo perché, eliminando le stringhe dalla lista, gli altri elementi scalano verso sx di una posizione mentre l'indice continua ad incrementare. Per risolvere questo problema bisogna modificare il codice. Potremmo, come soluzione, decidere di muoverci effettuando il controllo delle singole stringhe da destra verso sinistra (con indice e step del comando **range** negativi).

```
for i in range(len(stringhe)-1,-1,-1):
    if 'th' in stringhe[i]:
        stringhe.pop(i)
print(stringhe)
assert stringhe == expected, f'Sbagliato'
```

→ Partiamo da len()-1 fino all'indice -1 con step -1 (da dx a sx)

Soluzione 2

Con il while controlla ogni volta l'operazione booleana. Quindi se eliminando un valore si modifica la lunghezza della lista con il while lo vede direttamente. Potrò quindi fare un if ed incrementare l'indice solo nel caso in cui NON elimino nulla.

```
i = 0
while i < len(stringhe): # il controllo del while con len() e' eseguito ogni volta
    if 'th' in stringhe[i]:
        stringhe.pop(i) # tolgo elemento i-esimo; gli elementi a dx di i-th shiftano tut
                        # quindi in questo caso NON devo incrementare i
    else:
        i+=1            # se non ho tolto, posso andare avanti
assert stringhe == expected
```

Soluzione 3

Creare una nuova lista in cui andare ad inserire di volta in volta SOLO gli elementi che non contengono 'th'. Tuttavia, con questa soluzione dobbiamo stare attenti, perché per la correzione dell'esercizio avviene guardando la lista iniziale e non la nuova che abbiamo creato. Sarà quindi fondamentale sovrascrivere la nuova lista che abbiamo creato all'interno della lista iniziale.

```
stringhe = ['python', 'th', 'thhe', 'the', 'thee', 'ttthtt', 'aatata', 't h', 'pippoth', 'the s
out = []
for i in range(len(stringhe)):
    if 'th' not in stringhe[i]: # aggiungo se th non presente
        out.append(stringhe[i])
stringhe[:] = out # semplicemente uso lo slicing che mi sostituisce il contenuto di stri
assert stringhe == expected
```

Fare molta attenzione nel creare le copie di liste/string:

New_lista = lista sono due elementi che puntano allo stesso spazio in memoria. Quindi non abbiamo creato effettivamente una copia di list e modificando lista o new_lista modificheremo anche l'altro.



New_lista = lista[:] → crea una copia di lista. new_lista è un nuovo elemento (totalmente staccato dalla lista iniziale) e vi inserisce dentro tutti i valori di lista dall'inizio alla fine



lista[:] = new_lista → sovrascrive a lista dall'inizio alla fine, tutti i valori di new_lista

17/10/2023

Ordinare con .sort e Sorted

Il trucco è semplice: il comando `.sort` modifica la lista IN PLACE mentre `sorted` ne crea una copia ordinata. Infatti il comando `.sort()` o `sorted` riordinano gli elementi di una lista in ordine crescente/alfabetico. Per funzionare devono aver definito da qualche parte come confrontare i dati all'interno della lista. Questi comandi funzionano su string, tuple e liste ma restituiscono sempre una lista come output.

```
L = [100, 5, -2, 27, -1, 1, 0, 999]
## Ordinamento crescente
L.sort()
# dopo questa istruzione la lista L è ordinata in maniera crescente
L
[-2, -1, 0, 1, 5, 27, 100, 999]
```

Per l'ordinamento decrescente abbiamo invece bisogno del comando `.reverse` (o `reversed`). Di base il comando è considerato come False, se lo si vuole attivare basta specificarlo = True.:

```
## Ordinamento decrescente
L = [100, 5, -2, 27, -1, 1, 0, 999]
# si attiva passando il parametro di default reverse=True
L.sort(reverse=True)
L
[999, 100, 27, 5, 1, 0, -1, -2]
```

Per stabilire quale fosse l'ordine crescente gli scrittori di python si sono basati sul valore numerico associato ad ogni carattere nella sua rappresentazione ASCII.

ord() e chr()

In codice ASCII ad ogni carattere è assegnato un determinato numero. Con il comando `ord('A')` ci dirà quale è il numero ASCII associato ad A. Con in comando `chr(65)` invece ci dirà il carattere associato al numero 65 in ASCII. `ord()` e `chr()` sono uno l'inverso dell'altro. Può tornare particolarmente utile ricordare che nella codifica numerica delle stringhe:

- ' ' (spazio) viene prima di A e a
- 'A' maiuscolo viene prima di 'a' minuscolo

```
ord(' '), ord('A'), ord('a')
(32, 65, 97)
```

Ordinamento tramite chiave parziale

Se volessimo ordinare in base a criteri diversi da quello alfabetico (per es. ordinare le parole dalla più corta alla più lunga) possiamo utilizzare la parola chiave `key`.

`Ls = sorted(L, key = len) → ordina in base alla lunghezza (len)`

In questo caso abbiamo utilizzato una funzione pre-esistente in python (cioè `len`) ma potremmo inserire come `key` anche funzioni da noi create.

Supponiamo per esempio di dover riordinare con 2 criteri: prima in ordine crescente per lunghezza e poi, a parità di lunghezza, ordinare in base al valore in ASCII degli elementi stessi. (che equivale all'ordine alfabetico/numerico)


```
# dovete pensare che la funzione usi
# elemento i-esimo nella comparazione
def len_and_value(elem):
    return len(elem), elem
    #1 criterio #2 criterio

Ls = sorted(L, key=len_and_value)
```

Fare MOLTA attenzione perché `reverse=True` cambia TUTTI i criteri. (si capisce meglio con l'esempio pari/dispari più avanti).

Se invece volessimo ordinare gli elementi dal più lungo al più corto basta returnare dalla funzione `-len`, così da cambiargli segno. Se utilizzassimo `reverse` invece cambieremmo anche l'ordinamento per valore. (quindi metterebbe la z prima della a).

Problema: (spesso questo all'esame da fastidio se è chiesto) 🤔 🔥

Data la stringa

```
L = ['gli', 'eroi', 'son', 'tutti', 'giovani', 'e', 'belli']
```

tornare la lista che contiene le parole in:

- ordine crescente della lunghezza
- in caso di parità di lunghezza, in ordine INVERSO lessicografico
 - (prima la Z della A)

```
L = ['gli', 'eroi', 'son', 'tutti', 'giovani', 'e', 'belli']
# L = [ (3,4,5), (40,50,100) ]

def len_and_value(elem):
    return len(elem), elem

# il meno su len "annulla" il reverse = True
# ma poi il reverse = True funziona sulle stringhe
# chiedo di ordinare in maniera decrescente
Ls = sorted(L, key=len_and_value, reverse=True)

# controllo la lunghezza
length = []
for i in Ls:
    length.append(len_and_value(i))
print(Ls, length, sep='\n')
```

19/10/2023

Esercizio Ordinamento

```
a_list = [1,4,5,2,3,6,7,9,8,10]
def pari(element: int):
    p= element%2
    return p

new_list = sorted(a_list, key=pari)
print(new_list)
```

i n questo modo ci mette prima i PARI e poi dispari senza considerare il lor valore

```
a_list = [1,4,5,2,3,6,7,9,8,10]
def pari_and_value(element: int):
    p= element%2
    return p, element

new_list = sorted(a_list, key=pari_and_value)
print(new_list)
```

in questo modo ci mette prima i PARI ordinati anche in base al valore (dal piu piccolo al più grande). per fare dal più grande al più piccolo dobbiamo giocare con il reverse

```
a_list = [1,20,30,4,50,6,7,8,9,]
def pari_and_value(element: int):
    p= element%2
    return -p, element

new_list = sorted(a_list, key=pari_and_value)
print(new_list)
```

in questo modo ordino prima i DISPARI ordinati anche in base al valore

```
a_list = [1,20,30,4,50,6,7,8,9,]
def pari_and_value(element: int):
    p= element%2
    return -p, element

new_list = sorted(a_list, key=pari_and_value, reverse=True)
print(new_list)
```

Il Reverse cambia TUTTi i criteri. IN questo caso andrà quindi a mettere prima i PARI in ordine DECRESCENTE

19/10/2023

Operatore Ternario

Consente di scrivere un if su una riga sola. Sintassi:

```
<valore_se_vero> if <condizione> else <valore_se_falso>
```

Per capirlo aiuta leggere codice come:

```
<questo> if <condizione è vera> else <quest altro>
```

List Comprehension

Spesso capita di risolvere problemi che seguono 3 pattern principali lavorando su una sequenza.

- **Trasforma** ogni elemento della sequenza
- **Riduci** la sequenza
- **Filtra** elementi della sequenza

Per prendere gli elementi a due a due possiamo usare l'operatore zip. Ma stessa cosa potrei fare raggruppando insieme 3 stringe. Se le stringhe avessero lunghezze diverse lo zip si interrompe al termine della stringa più corta.

In generale la list comprehension e l'operatore ternario sono più veloci del for classico.

26/10/2023 (Probabilmente sarà all'esame)

Cercare una stringa dentro un'altra SLIDING WINDOW

- Questo problema è bene sapere come si risolve in quanto **si ripete moltissime volte**.
- E' un problema astratto: può essere applicato anche ad altre strutture dati
 - Ad esempio la stessa identica cosa può essere applicata alle immagini su strutture bidimensionali.
 - Questo è stato chiesto all'esame a Luglio (cerca una piccola immagine in una immagine grande).
- Potete pensare di cercare un qualcosa **query** dentro un qualcosa di più grande **corpo**

Data una stringa "query" in input e un'altra stringa "corpo", e' necessario trovare tutte le volte che "query" è contenuta nella stringa "corpo"

```
query = 'pippo'  
corpo = 'pippippopipipipipppppppippo'
```

```
def count_sub_string(query, corpo):
    offset, count = len(query), 0
    # mi fermo non appena query inizia
    # a sbordare dal corpo. Mi serve il +1
    # perche' altrimenti non controllo ultimo
    # elemento quando faccio slicing [start:end]
    N = len(corpo) - offset + 1
    out = ''
    for i in range(N):
        pquery = query
        if query == corpo[i:i+offset]:
            count += 1
            pquery = query.upper()
        line = str(i).zfill(2)+' '+'*i+pquery
        out += line + ' '* (len(corpo)+3-len(line))+'|\n'
    return out
```

Funzioni Lambda

Le funzioni Lambda, anche dette funzioni anonime, possono essere definite "al volo", molto rapidamente senza neanche il bisogno di definire un nome della funzione. Vengono usate spesso come chiave key nell' ordinamento per specificare ordinamenti parziali.

```
L = [
    #nome      cognome      voto      eta
    ('mario' , 'rossi' , 23 , 24),
    ('mario' , 'ferro' , 30 , 21),
    ('mario' , 'rossi' , 19 , 32),
    ('frank' , 'bianc' , 27 , 24),
    ('marta' , 'veri' , 25 , 29),
    ('mario' , 'rossi' , 21 , 32),
```

Supponiamo ora di voler riordinare la lista qui sopra in **ordine crescente per età**. Possiamo utilizzare una funzione rapida lambda:

```
S = sorted(L, reverse=False, key=lambda elem: elem[3] ) #elem[3] indicizza eta'
```

Ora voglio ordinare prima per:

1. **Cognome,**
2. **Nome**
3. **Età in ordine decrescente**
4. **Infine per voto in ordine crescente**

[illegible]

31/10/2023

Come fare un Istogramma tramite Dizionari

1. Modellazione – attraverso counts o get_histogram(L)

Riceviamo in ingresso una lista di interi → $L = \text{list}[\text{int}]$

1. Creiamo un dizionario counts che mappa il conteggio di ogni elemento della lista.
2. Iteriamo sugli elementi di L e controlliamo:
 - SE l'elemento NON è nel dizionario è la prima volta che lo vedo, quindi lo aggiungo al dizionario con conteggio 1.
 - SE invece l'elemento è già nel dizionario incremento il suo conteggio.
3. A questo punto ritorno il dizionario counts in cui: la cui chiave-elemento mi rende il contatore per quell'elemento.

```
#####  
## Prima Parte - Modellazione  
#####  
  
def get_histogram(L):  
    counts = {}  
    # creo un dizionario dove ogni intero in ingresso mappa al suo conteggio  
    for k in L:  
        # get fa sì che se la chiave c'è la rende  
        # se la chiave non c'è, rende 0  
        # vi evita di scrivere if v in counts else etc.  
        counts[k] = counts.get(k, 0) + 1  
        # se k è in counts allora rendo counts[k]  
        # se K non è in counts allora rendo 0  
    # restituisco la stringa che mi codifica il dizionario come  
    # stringa/tabella con sotto funzione hist_dict_to_str  
    return counts
```

`Dict.get(k, 0) + 1` recupera il valore associato a k. Se non lo trova va avanti con le operazioni (cioè ci somma 1), altrimenti svolge la condizione tra parentesi (in questo caso gli attribuisce il valore 0).

2. Visualizzazione, costruzione stringa

1. Selezionare il minimo e il massimo dalle chiavi. Itero su `range(minimo, massimo+1)`
2. Per ciascun valore del `range(min, max+1)`:
 - SE il valore NON è in counts vuol dire che è a frequenza 0 (non appare mai in L).
 - SE, invece, il valore è già in counts estraggo la sua frequenza da counts.
3. A questo posso passare ad assemblare la stringa.

Per svolgere tutti questi passaggi andiamo a “spezzettare” il problema in due funzioni:

```

def hist_dict_to_str(counts):
    # poiche' l istogramma deve anche avere i valori che non
    # compaiono in `values` allora ottengo anche il minimo e massimo
    # maxx, minn = max(counts.keys()), min(counts.keys())
    # i dizionari spesso espongono le chiavi quindi la linea sopra funziona
    # anche con:
    maxx, minn = max(counts), min(counts)

    # creo stringa vuota
    hist_table = str()
    for elem in range(minn, maxx+1): # massimo incluso
        # prealloco a zero
        n_times = 0
        # se i e' nel dizionario
        # prendo al sua frequenza
        # altrimenti e' gia' zero
        if elem in counts:
            n_times = counts[elem]
        # assemblo la stringa e la aggiorno
        # importante \n finale per andare accapo
        hist_table += str(elem) + "\t" + "*" * n_times + '\n'
    return hist_table

```

```

# assemblo tutto insieme
def int_to_hist(L):
    counts = get_histogram(L)
    return hist_dict_to_str(counts)

```

Tuttavia questo codice funziona solamente per una lista L di numeri interi. Se volessimo adattare il codice affinché funzioni anche con le stringe bisogna adottare alcuni cambiamenti nella prima parte di codice (lasciando tutto il resto uguale).

```

def hist_dict_to_str(counts):
    ## ----- Codice che gestisce anche le stringhe ----- ##
    # Codice per gestire anche stringhe
    a_key = list(counts.keys())[0]
    if isinstance(a_key, int): # type(a_key) == int
        maxx, minn = max(counts.keys()), min(counts.keys())
        elements = range(maxx, minn+1)
    else:
        elements = counts.keys()
    ## ----- Fine Codice che gestisce anche le stringhe ----- ##

    # creo stringa vuota
    hist_table = str()
    for elem in elements:
        # prealloco a zero
        n_times = 0
        # se i e' nel dizionario
        # prendo al sua frequenza
        # altrimenti e' gia' zero
        if elem in counts:
            n_times = counts[elem]
        # assemblo la stringa e la aggiorno
        # importante \n finale per andare accapo
        hist_table += str(elem) + "\t" + "*" * n_times + '\n'
    return hist_table

```

Per controllare il tipo possiamo o utilizzare il comando `type() == int/str/list etc.` oppure il comando `isinstance(variabile, tipo da controllare)` e restituirà `True/False`.

Anagramma

Per vedere se una stringa è l'anagramma dell'altro possiamo prendere i singoli elementi di ogni stringa e riordinarli con sorted. Se abbiamo due stringhe ordinate uguali allora sono una l'anagramma dell'altra.

```
sorted('a gentleman') == sorted('elegant man') ???
```

Tabella/Rubrica/Agenda

Supponiamo di voler creare un'Agenda, definita come una lista di dizionari, contenente tutti i contatti con relativi dati e n. di telefono, e di voler rappresentare tale agenda in una sorta di Tabella. Procederemo ovviamente per fasi attraverso 2 funzioni: una prima funzione per creare la struttura della tabella ed una seconda funzione per inserirci i dati dell'agenda. In generale possiamo dire che ogni chiave del dizionario è una colonna ed ogni record una lista

PRENDERE SCREEN E PROVARE IL CODICE IN SPYDER

Supponiamo ora di voler ordinare l'Agenda in base ai valori contenuti in una colonna specifica, nel nostro caso Cognome:

```
agenda_ordinata = sorted(agenda, key=lambda record : record['cognome'])
```

Se invece avessimo un ordinamento con più chiavi parziali

```
agenda_ordinata = sorted(agenda, key=lambda diz: [diz[c] for c in ord_columns if c in diz])
```

02/11/2023

Ricerca Lineare

Ricerca in UN Campo

Ora, immaginiamo di cercare il primo record la cui colonna (che indicheremo con campo) contiene il valore_cercato. Per ogni record nella agenda:

- Se il campo in cui dobbiamo cercare è nel record E <campo> == <valore_cercato> → ritorniamo il record (NB: è il primo trovato per costruzione)
- Se non trovo niente → ritorniamo None.

```
def find_record_by_valore(agenda : Agenda, campo : str, valore_cercato : str)
-> Record | None:
    for record in agenda:
        # Importante: prima controlla che CAMPO sia in record
        # e solo DOPO accedo
        if campo in record and record[campo] == valore_cercato:
            return record
    return None # possiamo anche ometterlo python torna
               # None se non mettiamo
```

Per ricercare TUTTI i record che soddisfano una determinata caratteristica in un campo specifico il procedimento è pressoché identico. In questo caso cerchiamo i record con cognome == "Mouse"

```
def find_all_records_by_valore(agenda, campo, valore):
    all_records = []
    for record in agenda:
        if campo in record and record[campo] == valore:
            all_records.append(record)
    return all_records

record_t = find_all_records_by_valore(agenda, 'cognome', 'Mouse')
```


Il tutto può essere scritto anche in una sola riga con la **list comprehension**:

```
def find_all_records_by_valore_lc(agenda, campo, valore):  
    return [record for record in agenda if campo in record and record[campo] == valore]
```

Ricerca in PIU' Campi

Se volessimo cercare, all'interno dell'agenda, TUTTI gli abitanti di Topolinia che fanno di cognome Gambadilegno?

- Riceveremo in input un dizionario contenenti le query
- Per il ragionamento è simile a quello di prima ma dobbiamo sviluppare una nuova funzione che controlla quando un record corrisponde alle query date.

```
query_multi_campo = { "città" : "Topolinia", "cognome": "Gambadilegno"}  
  
record = {'nome': 'Paperino', 'cognome': 'Paolino', 'telefono': '555-1313',  
          'indirizzo': 'via dei Peri 113', 'città': 'Paperopoli'}  
  
def match_query(record, query):  
    for campo, valore in query.items(): # per ogni coppia della query campo : valore_da  
        # se campo non e' nel record  
        # o se c'e' il campo, non corrisponde il valore  
        # torno subito False, non importa continuare a cercare  
        if campo not in record or record[campo] != valore:  
            return False  
    return True # se non torno mai vuol dire che c'e' un match  
  
# match_query(record, query_multi_campo) # invocazione
```

Il tutto può essere scritto anche in una sola riga con la **list comprehension**:

```
def find_all_records_by_multi_value_lc(agenda, query):  
    return [record for record in agenda if match_query(record, query)]
```

Ricerca Binaria

La ricerca lineare ha complessità $O(N)$ cioè direttamente proporzionale al n. di elementi su cui devo iterare. Esiste però un metodo più veloce nel caso in cui dovessi fare tante ricerche su un campo specifico (nel nostro caso città):

1. Ordinare l'agenda per il campo città → Complessità: $O(N \log_2(N))$ → Tuttavia verrà svolta una sola volta.
2. Fare una RICERCA BINARIA sul campo città → Complessità: $O(\log_2(N))$

In generale possiamo dire che con la ricerca binaria ci piazziamo al centro della lista da controllare e vediamo se $L[\text{half}] == \text{query}$. Successivamente ci spostiamo a destra per effettuare lo stesso controllo e se non troviamo nulla ci spostiamo a sinistra.

```
def bin_search(L, query):  
    # stampa L  
    print(L)  
    # se vuota, non lo abbiamo trovato  
    if not L:  
        return False  
    # dividi per 2  
    half = len(L)//2  
    # controlliamo nel mezzo  
    if L[half] == query:  
        print(f'{L[half]}!')  
        return True  
    # controllo a sx  
    elif query < L[half]:  
        return bin_search(L[:half], query)  
    # controlla a dx  
    else:  
        return bin_search(L[half+1:], query)
```


Indice Invertito

Esiste un modo ancora più rapido della Ricerca Binaria. Questo è il caso dell'Indice Inverso che ci torna utile dobbiamo cerca ogni volta in campi diversi. Per indice intendiamo una sequenza ordinata di valori, che associa a ciascun valore l'indice del record che contiene quel valore. Creare un Indice significa quindi creare un Dizionario che mappa {valore : lista_di_indici} per ogni record dell'Agenda:

```
def crea_indice(agenda, campo):
    indice = {} # questo mappa valore --> lista_indice
                # Masi ---> [0,1]
    for idx, record in enumerate(agenda):
        if campo in record:
            # accedo al valore
            valore = record[campo]
            # se non e' nell'indice
            if valore not in indice:
                # creo la lista con quel indice
                indice[valore] = [idx]
            else:
                # se e' gia, aggiorno la lista con nuovo indice
                indice[valore].append(idx)
    return indice
```

- Estraggo il <valore> dal record tramite il <campo> in cui si trova
- Se il <valore> NON è nell'indice, aggiungo la sua posizione.
- Se il <valore> è nell'indice, appendo la nuova posizione in fondo alle precedenti.

Andiamo ora a creare una tabella che contiene gli elementi di Indice:

```
def indice_to_table(indice):
    if not indice:
        return '# Indice Vuoto'
    max_cols = max(len(L) for L in indice.values())+1
    header = "| valore | indici |"
    header += "|" * (max_cols-1)
    header += '\n'
    header += '|---|'*max_cols+'|\n'
    for valore, linea in indice.items():
        header += f"| {valore} |"
        for i in range(max_cols):
            if i < len(linea):
                header += f"| {linea[i]} |"
            else:
                header += " |"
        header += '\n'
    return header
```

valore	indici	
Paolino	0	
Paperone	1	
de' Paperoni	2	
Pitagorico	3	
Gambadilegno	4	5
Mouse	6	7
de' Pippis	8	

Vediamo ora un esercizio in cui dato Indice, tornare tutti i record di Mouse:

```
def records_per_campo(indice, agenda, campo):
    return [ agenda[idx] for idx in indice[campo]]

partial_records = records_per_campo(indice, agenda, 'Mouse')
```

	nome	cognome	telefono	indirizzo	città
0	Topolino	Mouse	555-12345	via degli Investigatori 1	Topolinia
1	Minnie	Mouse	555-54321	via di M.me Curie 1	Topolinia

07/11/2023 (Dalle slides)

ggg

Manipolazione File (vedere anche HW4req)

r → **Read**: Apre il file per leggere, se il file non esiste da errore

a → **Append**: Apre il file per appendere qualcosa (aggiunge in fondo), se il file non esiste lo crea

w → **Write**: Apre il file per scrivere, se non esiste crea il file

x → **Create**: Crea il file se non esiste, se esiste da errore.

Per lavorare con i file abbiamo due opzioni. Possiamo aprirli e assegnarli direttamente ad una variabile, ma in questo caso andranno necessariamente CHIUSI, oppure possiamo utilizzare il comando `with open()` che automaticamente (tramite indentazione) svolge le operazioni indicate e chiude il file al termine.

Read

`with open("nome del file", "r")` → apre il file in `f` come "solo lettura"

`f` potrà poi essere manipolato con appositi comandi come `f.readline()` che legge le righe del file. Questo comando si aggiorna costantemente, ciò significa che se stampo 5 volte `f.readline` la prima volta stampa la prima riga, la seconda volta la seconda, la terza la terza e così via...

ESEMPIO CODICE

Per iterare sulle singole righe si potrebbe anche fare un banalissimo ciclo `"for riga in f:"`

Esempio codice

Append

`With open("nome del file", "a")` → apre il file in `f` come "solo lettura"

Write

GGG

Create

Copiare testi da un file ad un altro

With `open('file1', 'r') as fr, open('file2', 'r') as fw:`

`fw.write(fr.read())`

09/11/2023

Programmazione Funzionale

Programmazione a Oggetti (Object Oriented)

Cos'è???

L'**ereditarietà** è un concetto secondo cui una classe (detta figlia) eredita da un'altra classe (detta madre) tutte le proprietà della classe madre e la estende (cioè ne aggiunge altre):

- Gli attributi comuni a tutti gli oggetti della classe vengono detti Attributi di Classe.
- Invece gli attributi propri che variano tra oggetto e oggetto vengono detti Attributi di Oggetto

Potremmo fare l'esempio degli esseri viventi. TUTTI gli esseri viventi (classe madre) respirano, mangiano, dormono etc. ma possono essere divisi in varie specie (in cui ogni specie è una classe figlia) con caratteristiche e proprietà diverse (e dette estese) rispetto alle proprietà comuni a TUTTI gli esseri viventi.

Potremmo fare l'esempio anche della classe madre Persona e poi andare a fare delle "sottoclassi" figlie per diverse professioni. Per esempio gli insegnanti avranno attributi e proprietà diverse dalle persone normali. Per esempio ogni insegnante ha una materia che insegna, o la possibilità di attribuire dei voti e così via:

Foto codice da Edoardo Minali (se lo facessi io sarebbe meglio)

Funzione super

```
1 class Alien:
2     """
3     Class that represents an Alien in a videogame and we need to keep track of how many Aliens
4     are present
5     """
6     # attributo di classe
7     n_instances = 0
8
9     # definizione di metodi di classe
10    @classmethod
11    def get_n_instances(cls):
12        return cls.n_instances
13
14    @classmethod
15    def increment_instances(cls):
16        cls.n_instances += 1
17
18    @classmethod
19    def decrement_instances(cls):
20        cls.n_instances -= 1
21
22    def __init__(self, name='et'):
23        self._name = f'{name}{self.get_n_instances()+1}'
24        self.increment_instances()
25        print(self)
26
27    def __repr__(self):
28        return f'[NEW BORN] Alieno {self._name} pronto a combattere'
29
30    def __del__(self):
31        print(f'[DEAD] Alieno {self._name} e\' morto')
32        self.decrement_instances()
```

Con il decoratore?? Possiamo fare dei classmetod. Nell'esempio degli alieni usiamo un classmetod che ci restituisce il numero di istanze di quella classe, e ci torna utile per sapere quanti oggetti appartengono a quella classe (cioè quanti alieni sono in gioco).

Questa cosa si può fare anche al contrario.

Decoratori

Un decoratore è una funzione (e quindi come sappiamo un oggetto) che modifica le «funzionalità» di una funzione, senza però modificare il codice. Supponiamo infatti di avere una funzione “bloccata” che non possiamo modificare. Il trucco è di ricordarsi che si possono passare le funzioni come parametri di altre funzioni:

```
# this is the function to be extended
def my_locked_function():
    print('Main code of the locked function 1')
```

```
# this is the decorator
def before_and_after_decorator(func):
    def my_wrapping_function():
        print('> Decorator: before')
        func()
        print('> Decorator: after')
    return my_wrapping_function
```

```
# this is the function to be extended
def my_locked_function():
    print('Main code of the locked function 1')

# my decorator is "templated" so that the func() is not decided a priori
# and will depend on what we pass as a function
dec = before_and_after_decorator(my_locked_function)
```

✓ et voilà~

```
1 dec()
> Decorator: before
Main code of the locked function 1
> Decorator: after
```

```
# puo' anche essere chiamato cosi
@before_and_after_decorator
def my_locked_function_two():
    print('Main code of the locked function 2')
```

Overloading operatori

L’overloading degli operatori consiste nel “potenziare” gli operatori base per fargli svolgere funzioni specifiche.

Per farlo andiamo praticamente a “sovrascrivere” delle funzioni base. In particolare se vogliamo modificare l’operatore + andiamo a ridefinire la funzione __add__.

Supponiamo di avere le solite tuple con valori RGB e di voler modificare l’operatore + affinché non ci concateni le due tuple ma bensì faccia la somma dei valori DENTRO le tuple:

```
def __add__(self, other_col):
    "overloading of the add operator to define what means summing up two colors"
    return Color(self._r+other_col._r,
                  self._g+other_col._g,
                  self._b+other_col._b)
```

```
1 c1 = Color(0,255,0)
2 c2 = Color(255,0,0)
3 print(c1 + c2)
```

Color (255, 255, 0)

```
1 c1._r + c2._r, c1._g + c2._g, c1._b + c2._b,
(255, 255, 0)
```

→ Invoca __add__

In alcuni casi si può “sollevare”, con raise, un determinato errore che verrà stampato nella console dell’utente che sta usando la nostra libreria.

```
def __mul__(self, k):
    "overloading of the mul operator to define what means multipling two colors"
    if isinstance(k,int):
        print('using scalar multiply')
        return Color(self._r*k, self._g*k, self._b*k)
    elif isinstance(k,Color): → se k è un colore, faccio elementwise multiply
        print('using elementwise multiply')
        return Color(self._r*k._r, self._g*k._g, self._b*k._b)
    else:
        raise TypeError(f'Only multiplying between a) two colors b) color and a scalar are allowed,'
                        f' but I got {type(k)}')
```

Funzioni come Oggetti

Ogni cosa in Python è rappresentato da un oggetto (tranne alcune parole chiave). Un oggetto per essere tale possiede: un'identità (puntatore alla memoria), un tipo, valore. In python, proprio perché hanno queste 3 caratteristiche, anche le funzioni sono oggetti:

```
In [15]: id(max), type(max), max
Out[15]: (140237918869920, builtin_function_or_method, <function max>)
```

Ciò significa che possiamo dare delle funzioni come parametri di altre funzioni.

```
In [21]: def myfunc(data,operator):
...:     return operator(data)
...:

In [22]: myfunc([1,2,3],max)
Out[22]: 3

In [23]: myfunc([1,2,3],min)
Out[23]: 1
```

Oppure possiamo returnare da una funzione un'altra funzione:

```
In [116]: def power_factory(exp=2):
...:     def power_func(num):
...:         return num ** exp
...:     return power_func
...:

In [117]: cube_func = power_factory(exp=3)

In [118]: square_func = power_factory(exp=2)

In [119]: cube_func(3) == 3**3
Out[119]: True

In [120]: square_func(5) == 5**2
Out[120]: True
```

Iteratori (Eager)

Iterator:

E' un oggetto che implementa un protocollo di iterazione per iterare su uno stream finito/infinito di dati.

1. `__iter__()`
2. `__next__()`
3. Raise `StopIteration`

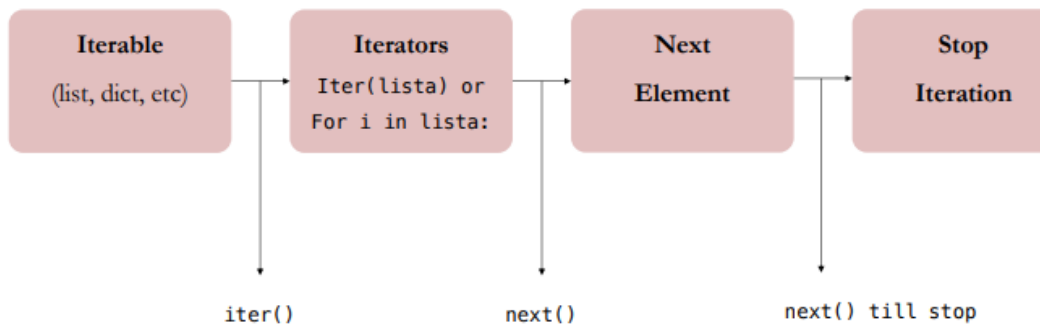
Iterable:

Ogni oggetto che può essere usato con un ciclo for (loop over it).

Un oggetto è *iterable* se vi fornisce un iterator come output.

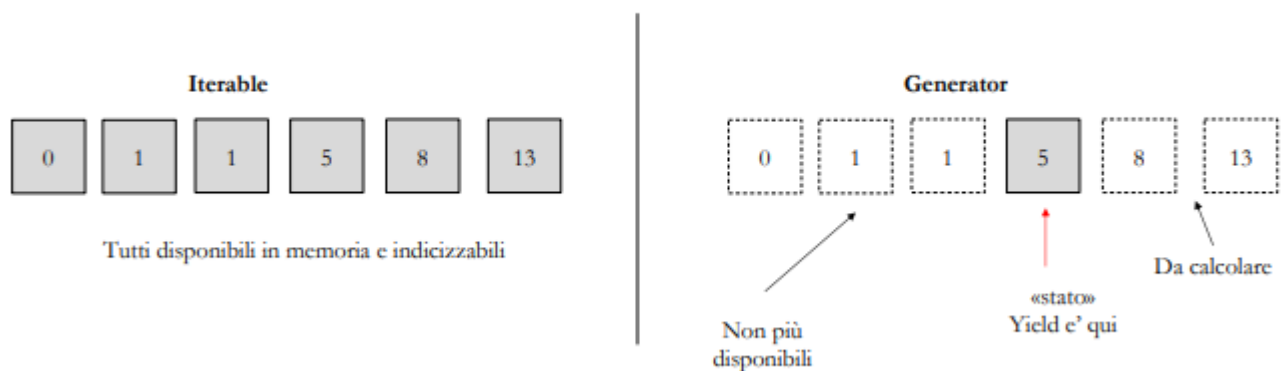
1. `__iter__()`
2. `__getitem__()`

gestisce
a[i]
indexing



Generatori (Lazy)

Sono definiti Lazy, cioè pigri, perché ci forniscono i dati uno alla volta. Tornano molto utili perché risparmiano spazio in memoria e impiegano meno tempo nel caso in cui dobbiamo lavorare con una grande quantità di dati.



Return chiude il flusso della funzione, torna immediatamente al main. **Yield** invece salva il risultato ma senza tornare al chiamante (?)

File .json

I file json permettono di passare facilmente da un linguaggio di programmazione ad un altro importando dei dati. I comandi principali sono:

- **import json**
- **Json.loads(x)** → “Importare” cioè convertire da json a python (str, list, dict etc...)
 - Foto Edo Minali
- **Json.dumps(struttura dati in cui vogliamo salvare, fw)** → “Esportare” dati
 - Foto da Ed Minali

Formattazione

Sempre con il comando `json.dumps()` possiamo andare a specificare l'indentazione o i "separatori", cioè i caratteri che dividono ogni riga da un'altra, oppure ogni chiave dal valore di un dizionario etc.

Foto edo minali

Ordinamento

Anche per l'ordinamento è sufficiente il comando `json.dumps()` andando ad indicare `sort/sorted??`

Suppoiamo per esempio di avere un dizionario e di volerlo ordinare in ordine alfabetico delle chiavi:

foto edo minali

14/11/2023

Mini Motore di Ricerca

Per fare un mini motore di ricerca abbiamo bisogno di due parti

- Una parte offline in cui bisogna costruire un istogramma normalizzato per ogni file. Per normalizzare l'istogramma:

ggg

- Una parte online ci viene fornita una stringa e dobbiamo ricercare quelle parole

Map (Trasforma)

La funzione Map implementa nella programmazione funzionale il Trasforma della programmazione oggetti. Sostanzialmente Map applica una funzione a tutti gli elementi.

$$[a_1, a_2, \dots, a_n] \mapsto [f(a_1), f(a_2), \dots, f(a_n)]$$

A livello di codice, prima definisco una funzione e successivamente con map applico tale funzione a tutti gli elementi:

```
>>> def upper(s):  
...     return s.upper()  
  
>>> list(map(upper, ['sentence', 'fragment']))  
['SENTENCE', 'FRAGMENT']  
>>> [upper(s) for s in ['sentence', 'fragment']]  
['SENTENCE', 'FRAGMENT']
```

Filter (Filtra)

La funzione Filter, “filtra” gli elementi, cioè implementa l’if a livello funzionale. Immaginiamo per esempio di avere i numeri da 1 a 10 e di voler prendere solo gli elementi multipli di due (cioè pari). Anche qui, prima definiamo la funzione e successivamente richiamiamo filter(func, elementi da filtrare):

```
>>> def is_even(x):  
...     return (x % 2) == 0  
  
>>> list(filter(is_even, range(10)))  
[0, 2, 4, 6, 8]
```

All e Any (AND e OR)

Sono la versione funzionale di AND e OR.

AND

All(False, False, True) → False

OR

Any(False, False, True) → True

16/11/2023

Deep e Shallow Copies

Esistono due modi di fare una copia:

Shallow copy: (src -> dst)

Crea una nuova variabile **dst**,
che referencia l'oggetto
puntato da **src**

Deep copy: (src -> dst)

Crea una nuova variabile **dst**,
che «copia» (ridondante) il
contenuto puntato da **src**

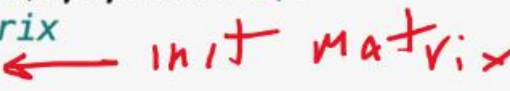
Metodo Shallow: semplicemente creiamo un oggetto che punta allo stesso spazio in memoria di src.

Metodo Deep: creiamo effettivamente una nuova variabile che contiene gli stessi elementi di src.

Matrici

Creare:

1. Creo il container esterno e lo chiamiamo matrix
2. Faccio un ciclo for che itera verticalmente creando ogni volta una riga
3. Faccio un ciclo for annidato per inserire in ogni riga i valori. In questo modo sto creando le colonne.
4. Dopo aver creato tutte le colonne per la prima riga inseriamo la riga dentro il container matrix (con append!)

```
def create_matrix(r,c,value=0):  
    # define matrix  
    matrix = []   
    # for each row  
    for each_r in range(r):   
        # define row  
        row = []  
        # for each col  
        for each_c in range(c):  
            # append the col to the row  
            row.append(value)   
        # append the row to the matrix  
        matrix.append(row)  
    return matrix
```

Se dobbiamo creare una matrice vuota possiamo farlo in maniera molto più rapida con solo due righe di codice (o addirittura una sola riga con generatore map):

```
def create_matrix_short_ok(r,c,value=0):  
    '''  
    a single for loop  
    '''  
    matrix = []  
    for each_r in range(r):  
        matrix.append([value] * c )  
    return matrix
```

```
def create_matrix_lc(r,c,value=0):  
    '''  
    with list comprehension  
    '''  
    return [ [value] * c for each_r in range(r) ]
```

```
def create_matrix_map(r,c,value=0):  
    '''  
    with map and lambda function  
    '''  
    return list(map(lambda each_row: [value] * c, range(r)))
```

Modo SBAGLIATO X

```
1 def create_matrix_short(r,c,value=0):
2     '''
3     implementation is not correct since
4     each row refers to the same row (shallow copy)
5     '''
6     return [ [value] * c ] * r
```

Questo metodo è sbagliato perché andremmo a creare una shallow copy della prima riga tante volte quanto è la grandezza dell'immagine. Il tutto è sbagliato perché, essendo una shallow copy, se modificiamo una riga verrebbero modificate anche tutte le altre.

Modo Rapido V

```
In [4]: n_Righe=10
In [5]: n_Colonne=3
In [6]: matrice=[[0,)*3]*n_Colonne for _ in range(n_Righe)]
In [7]: matrice
Out[7]:
[[0, 0, 0], [0, 0, 0], [0, 0, 0]],
[[0, 0, 0], [0, 0, 0], [0, 0, 0]],
[[0, 0, 0], [0, 0, 0], [0, 0, 0]],
[[0, 0, 0], [0, 0, 0], [0, 0, 0]],
[[0, 0, 0], [0, 0, 0], [0, 0, 0]],
[[0, 0, 0], [0, 0, 0], [0, 0, 0]],
[[0, 0, 0], [0, 0, 0], [0, 0, 0]],
[[0, 0, 0], [0, 0, 0], [0, 0, 0]],
[[0, 0, 0], [0, 0, 0], [0, 0, 0]],
[[0, 0, 0], [0, 0, 0], [0, 0, 0]]
```

Controllare le Dimensioni

Per controllare l'altezza e la larghezza di una matrice ci è sufficiente andare a vedere il numero delle righe e il numero delle colonne. Tuttavia abbiamo già visto che la matrice non è altro che una lista di liste... quindi quali sono le righe e quali le colonne?

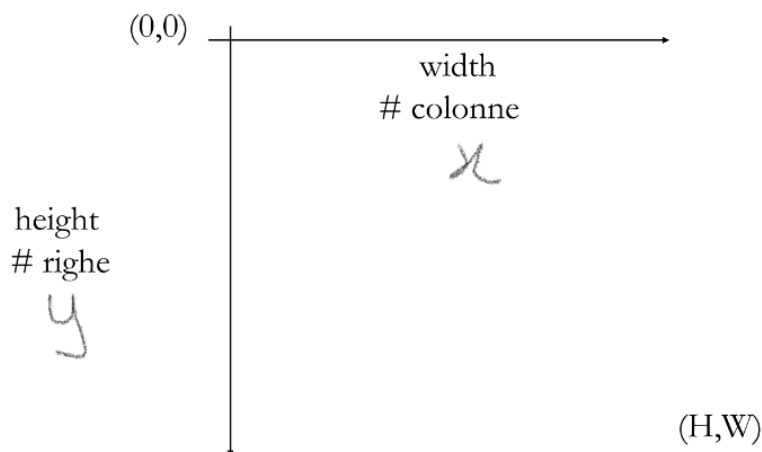
Ogni lista è una riga ed ogni elemento all'interno di ogni lista è una colonna.

```
[[2, 0, 0],
 [0, 4, 0],
 [0, 0, 5]]
```

```
def shape(mat):
    # immediatly check empty matrix
    if len(mat) == 0:
        return 0, 0
    if len(mat[0]) == 0:
        return 1, 0
    # rows corresponds to height
    r = len(mat)
    c = len(mat[0])
    return r, c
```

Sistema di riferimento

Fare molta attenzione perché gli assi sono orientati diversamente da quanto abbiamo sempre visto con il piano cartesiano:



Disegnare Rette

```
# maps names to colors in tuple
colormap = { 'red': (255,0,0), 'blu': (0,0,255), 'green': (0,255,0),
             'black': (0,0,0), 'white': (255,255,255)
            }
```

```
color_mat = create_matrix_lc(256, 256, colormap['black'])
```

x,y length

A horizontal line segment representing a row in the matrix. The starting point is labeled 'x,y' and the length is labeled 'length'.

x,y

length

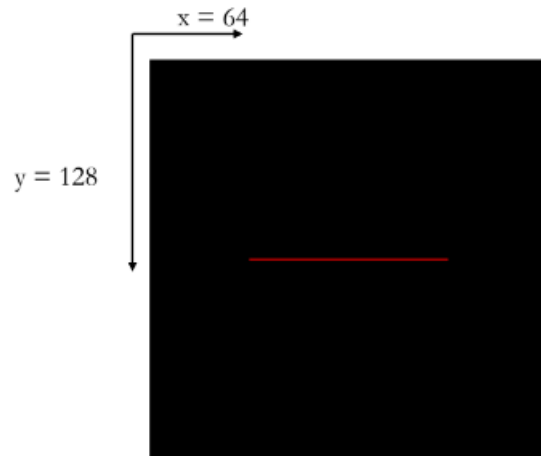
Per le righe (Retta Orizzontale)

Essendo una matrice formata da una lista per ogni riga, per disegnare una riga dritta ci basta accedere alla lista corrispondente alla riga e sovrascrivere i dati che c'erano dentro utilizzando lo slicing. Ci spostiamo dal punto x al punto x + length.

```
def plot_line_h(mat, x, y, length, value):
    """
    ricordarsi che mat
    e' indicizzata [riga][colonna] oppure [y][x]
    """
    mat[y][x:x+length] = [value] * length
```

Esempio

```
1 color_mat = create_matrix_lc(256, 256, colormap['black'])
2 *line_h, = 64, 128, 128
3 plot_line_h(color_mat, *line_h, colormap['red'])
```



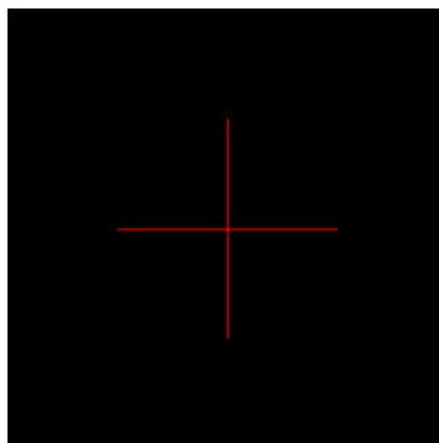
Per le colonne (Retta Verticale)

Per le colonne è più complicato, perché queste non sono una singola lista. Quindi non possiamo usare lo slicing ed abbiamo bisogno di utilizzare dei cicli for.

```
def plot_line_v(mat, x, y, length, value):
    '''
    ricordarsi che mat
    e' indicizzata [riga][colonna] oppure [y][x].
    ci muoviamo su y;
    andiamo da y fino a y+length, x rimane fissa
    '''
    for each_y in range(y, y+length):
        mat[each_y][x] = value
```

Esempio

```
1 plot_line_v(color_mat, 128, 64, 128, colormap['red'])
2 Image(color_mat)
```



Disegnare Rettangoli

Dobbiamo semplicemente disegnare 4 righe dritte:

```
def draw_rectangle(im, riga, colonna, h_rectangle, w_rectangle, colore):
    #SCRIVIAMO SULLE COLONNE (linee orizzontali)
    for x in range(colonna, colonna+w_rectangle):
        im[riga][x] = colore
        im[riga+h_rectangle][x] = colore

    #SCRIVIAMO SULLE RIGHE (linee verticali)
    for y in range(riga, riga+h_rectangle):
        im[y][colonna] = colore
        im[y][colonna+w_rectangle] = colore

    return im
```

Riga = riga da cui dobbiamo partire

Colonna = colonna da cui dobbiamo partire

H_rectangle = altezza del rettangolo

W_rectangle = larghezza del rettangolo

E se il Rettangolo finisse fuori dall'immagine?

Metodo 1

- Prima di tutto dobbiamo andarci a calcolare quale sarà l'ultima riga da scrivere per evitare che iterando con il ciclo for andremo in IndexError Out of range. Questa sarà uguale al minimo tra il punto in cui vogliamo arrivare e l'altezza o larghezza dell'immagine.
- Controllare che le coordinate in cui stiamo cercando di scrivere siano all'interno dell'immagine (cioè compresi tra 0 e la larghezza o altezza). Il caso affermativo andremo a colorare, altrimenti no.

```
def draw_rectangle_masi(im, riga, colonna, h_rectangle, w_rectangle, colore):
    H, W = shape(im)
    #SCRIVIAMO SULLE COLONNE (linee orizzontali)
    ultima_colonna = min((colonna+w_rectangle), len(im[0]))
    for x in range(colonna, ultima_colonna):
        im[riga][x] = colore
        if 0 <= x < W and 0 <= riga+h_rectangle < H:
            im[riga+h_rectangle][x] = colore

    #SCRIVIAMO SULLE RIGHE (linee verticali)
    ultima_riga = min((riga+h_rectangle), len(im))
    for y in range(riga, ultima_riga):
        im[y][colonna] = colore
        if 0 <= colonna+w_rectangle < W and 0 <= y < H:
            im[y][colonna+w_rectangle] = colore

    return im
```

```
# --- controllando le posizioni
def draw_pixel2(img, x, y, colore):
    altezza = len(img)
    larghezza = len(img[0])
    if 0 <= x < larghezza and 0 <= y < altezza:
        img[y][x] = colore
```

(È necessario che il punto di partenza sia all'interno dell'immagine. (anche se termina fuori))

Metodo 2 - ("Brutto")

```
# --- usando try-except per catturare l'errore
def draw_pixel(img, x, y, colore):
    try:
        img[y][x] = colore
    except IndexError:
        pass
```

Attenzione ai valori negativi: **NON** danno errore, ma si va al contrario!

Attenzione a non mettere except Exception con Errore **TROPPO** generico

Riempire l'area di un Immagine (Tipo Secchiello)

Iterativamente

L'idea è quella di trovare tutti i pixel neri vicino a noi per poi colorarli. Partiamo quindi dalle coordinate che ci vengono date in input e andiamo controllare il pixel sopra, sotto, a destra e a sinistra: se questo è nero (o comunque dello stesso colore dello sfondo) DOBBIAMO colorarlo. Se invece è di un colore diverso dallo sfondo significa che abbiamo incontrato il lato di un quadrato/rettangolo e dobbiamo fermarci.

A livello sostanziale: ogni volta che incontriamo un punto da colorare salviamo le coordinate e lo appendiamo in una lista. All'iterazione successiva andremo a colorare quel pixel e lo rimuoviamo dalla lista con il comando lista.pop()

Essendo che dobbiamo restituire l'area della figura che abbiamo colorato basterà utilizzare una variabile counter che incrementa ogni volta che coloriamo un nuovo pixel. Alla fine quindi avremo il conteggio di quanti pixel abbiamo colorato.

```
def riempi_iter(im, riga, colonna, colore=(255,255,255)):
    pixel_da_colorare = [(riga, colonna)]
    counter = 0
    #Il primo pixel da colorare lo diamo noi in input e poi proseguiamo
    #finchè ci sono pixel_da_colorare
    while len(pixel_da_colorare) > 0:
        #Prendiamo le coordinate del pixel da colorare
        riga, colonna = pixel_da_colorare.pop()
        #poi le cancelliamo con .pop() perchè le andremo a colorare

        #CONTROLLIAMO IL PUNTO IN CUI CI TROVIAMO
        if im[riga][colonna] == (0,0,0):
            im[riga][colonna] = colore
            counter += 1

        #CONTROLLIAMO IL PIXEL SOPRA
        if riga > 0 and im[riga-1][colonna] == (0,0,0):
            pixel_da_colorare.append((riga-1, colonna))
        #CONTROLLIAMO IL PIXEL SOTTO
        if riga < len(im)-1 and im[riga+1][colonna] == (0,0,0):
            pixel_da_colorare.append((riga+1, colonna))

        #CONTROLLIAMO IL PIXEL A SINISTRA
        if colonna > 0 and im[riga][colonna-1] == (0,0,0):
            pixel_da_colorare.append((riga, colonna-1))
        #CONTROLLIAMO IL PIXEL A DESTRA
        if colonna < len(im[0])-1 and im[riga][colonna+1] == (0,0,0):
            pixel_da_colorare.append((riga, colonna+1))

    return counter #Fuori dal ciclo while!!!
```

CONTROLLAREEE!!

Ricorsivamente

Un po' sulle immagini

Le immagini possono essere **Raster**

Griglia che quantizza i colori

Formati: jpeg, png, tiff

...oppure **vettoriali**

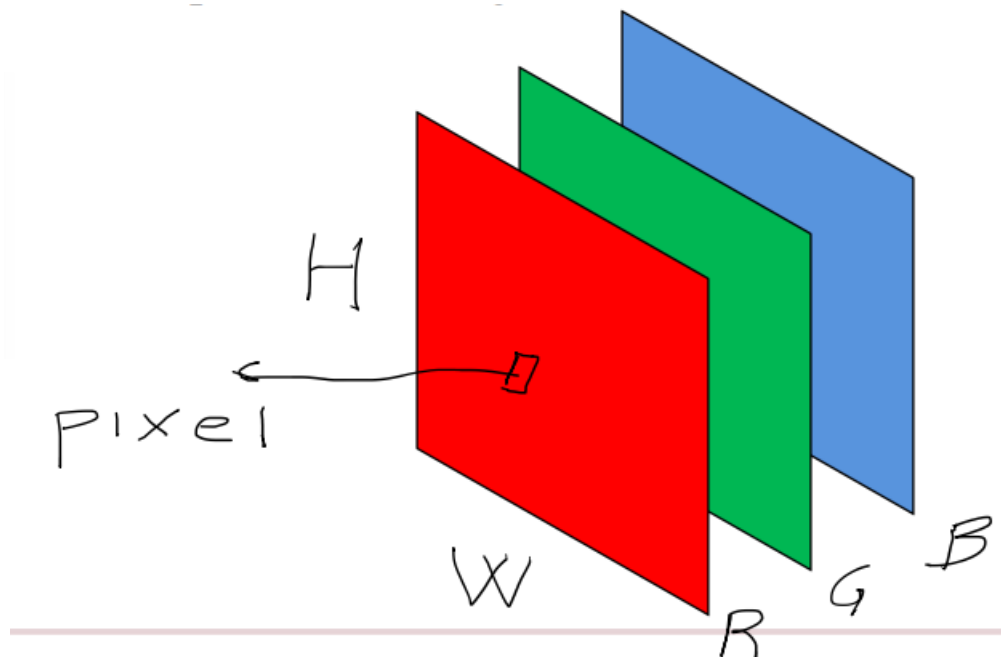
Curve e tracciati matematici che descrivono l'immagine

Formati: svg, eps, pdf

Le immagini (raster) possono essere rappresentate come:

1. Scala di grigi (quelle viste fin ora) **lista di liste di valore in scala di grigi**
2. RGB (a colori): **lista di liste di tuple che indicano i canali R,G,B**

Lavorare con le immagini

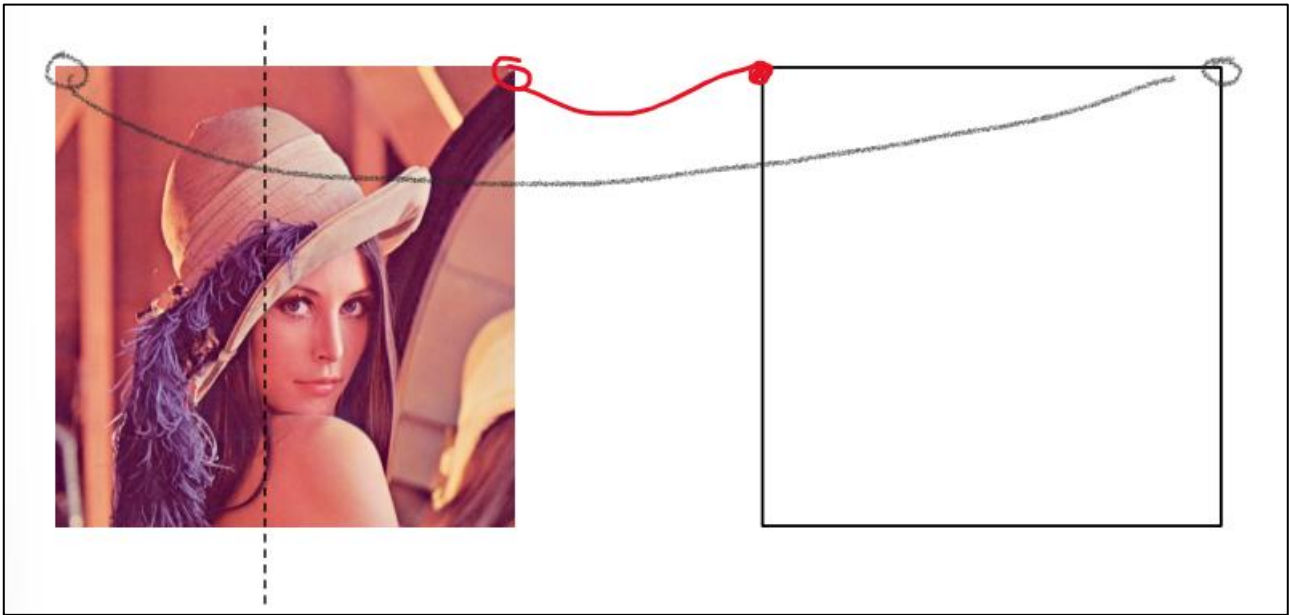


Le immagini per noi non sono altro che una matrice, cioè una lista di liste contenente come una serie di tuple che rappresentano i singoli pixel. Ogni tupla avrà poi tre valori, che sono rispettivamente RedGreenBlue e che ci permetteranno di rappresentare visivamente tutti i colori. Ogni valore della tupla può andare da 0 a 255. (la tupla (0,0,0) sarà il nero ed la tupla (255,255,255) il bianco).

Per lavorare facilmente con le immagini possiamo **importare il modulo images**. Fondamentali sono i comandi:

- `images.load("path")` → carica l'immagine nel file
- `images.visd(im)` → stampa a video l'immagine
- `images.Image([[(valori dei pixel) for _ in range()]] for _ in range())` → crea e stampa a video
- `image.save(im, path)`

Flippare un'immagine (Verticalmente)



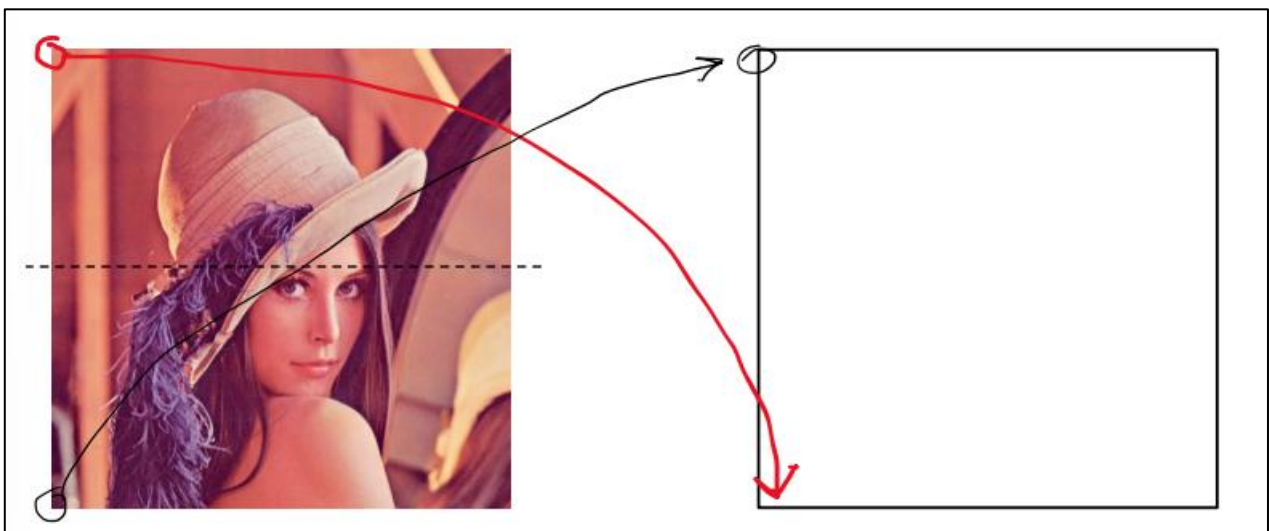
Rispetto all'Asse Verticale basta capovolgere ogni riga. Essendo che ogni riga è una lista possiamo tranquillamente usare l'operatore di slicing. `Image_flipped = [ r = r[::-1] for r in image]`

```
1 import images

1 lena = images.load('lena.png')

1 def flip_v(img):
2     H, W = shape(img)
3     flipped_img = []
4     for each_y in range(H):
5         flipped_img.append(img[each_y][::-1])
6     return flipped_img
7
8 def flip_v_map(img):
9     H, W = shape(img)
10    flipped = map(lambda each_r: each_r[::-1], img)
11    return list(flipped)
```

Flippare un'immagine (Orizzontalmente)



Rispetto all'Asse Orizzontale basta capovolgere tutto il container, cioè la matrice stessa. `Im_flip = im[::-1]` (possiamo anche utilizzare il generatore `reversed` ma bisogna stare attenti con i tipi. Infatti dovremo fare `list(reversed(im))` per assicurarci di passare da type generatore a list).

```
1 def flip_h(img):
2     H, W = shape(img)
3     flipped_img = []
4     for each_y_flip in reversed(range(H)): #range(H-1,-1,-1):
5         flipped_img.append(img[each_y_flip][:])
6     return flipped_img
7
8 def flip_h_map(img):
9     return list(map(lambda each_rr: each_rr, reversed(img)))
```

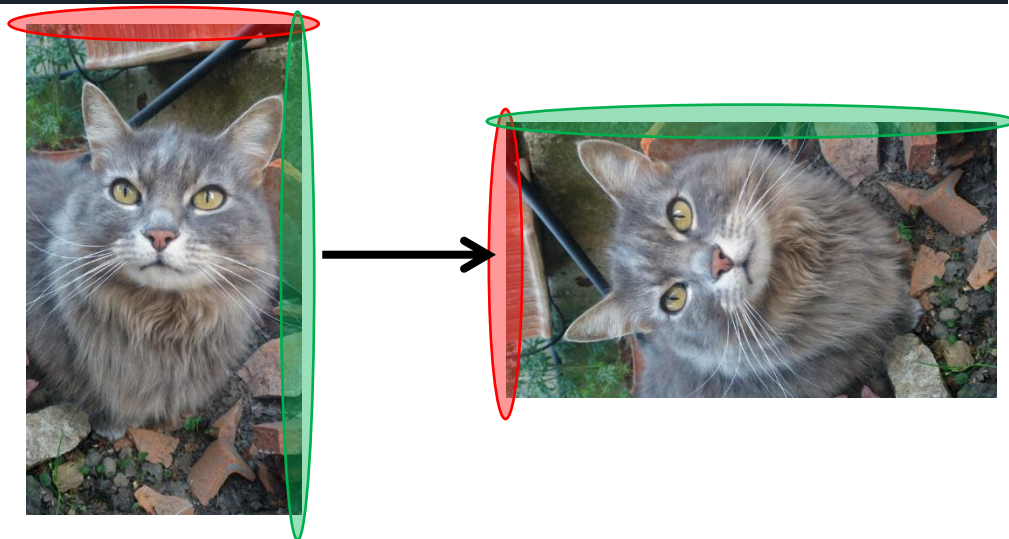
Ruotare un'immagine (di 90° verso sx)

Per ruotare un'immagine basta invertire righe e colonne, dunque fare in modo che la prima riga diventi la prima colonna. Tuttavia, come abbiamo già imparato nel corso, le matrici/immagini sono scritte a partire dalle righe. Risulta quindi più semplice pensare che dobbiamo trasformare le colonne in righe riga.

In generale la domanda da farsi è: Dove devo andare a prendere la prima riga???

Per ruotare verso sinistra conviene partire dalla colonna più a destra (cioè l'ultima colonna) per renderla la riga più in alto (cioè la prima riga). Per partire dall'ultima colonna usiamo `reversed()`:

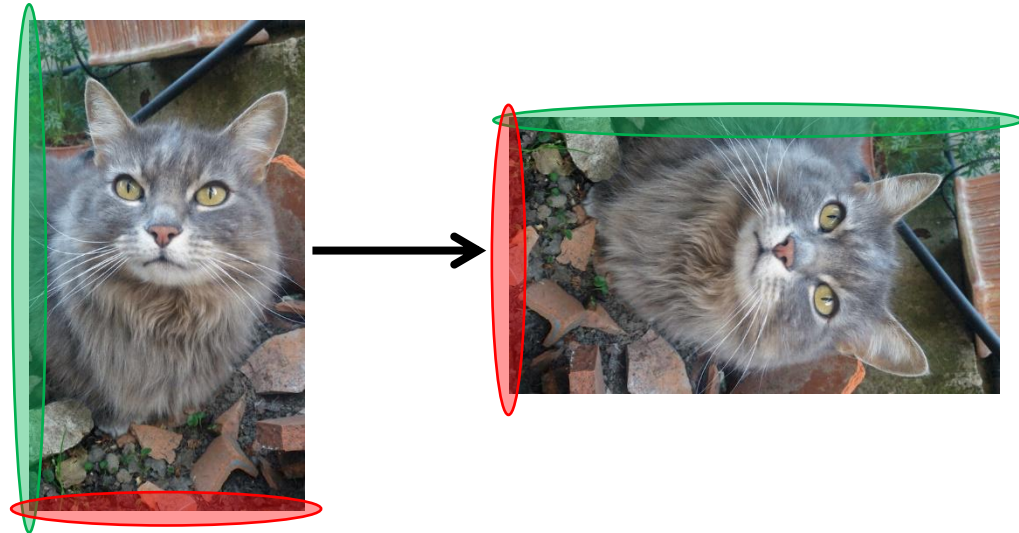
```
def rotate_left(im):
    return [ [im[r][c] for r in range(H)] for c in reversed(range(W))]
```



Ruotare un'immagine (di 90° verso dx)

Per la rotazione a destra l'idea è esattamente la stessa, anzi, dovrebbe risultare anche più semplice siccome possiamo andare a prendere direttamente la prima colonna e renderla la prima riga. La funzione dovrebbe essere la seguente:

```
def rotate_right(im):
    return [[im[r][c] for r in reversed(range(H))] for c in range(W)]
```



Ritagliare un'immagine (Crop)

Per ritagliare un'immagine è semplice, basta andare a prendere solo una porzione della stessa. Quindi definiamo una funzione che prende come default "l'area" che ci interessa e combiniamo il tutto utilizzando operatore di slicing sulle liste che rappresentano righe/colonne.

```
def crop(im, x, y, w, h):
    H, W = shape(im)
    return [ [c for c in row[x:x+w]] for row in im[y:y+h]]
```

Concatenare Due Immagini



Con zip possiamo andarci a prendere le righe a due a due e concatenare insieme attraverso l'operatore + (siccome sono liste)

```
def concatenare(img_list):
    # TODO: concatena sulle righe
    # NON appena si finisce la N-esima riga di TUTTE le matrici scrive nuova riga
    return [ [v for row in rows for v in row] for rows in zip(*img_list) ]
```

Applicare Filtri - Scala di grigi

```
# per trasformare una immagine in livelli di grigio
def gray(im):
    return [ [ (sum(c)//3,)*3 for c in row] for row in im ]
```

Applicare Filtri - Aumento Luminosità

```
def luminosita(pix,k):  
    def clip(p):  
        return max(min(int(round(p*k)),255),0)  
    return tuple(clip(p) for p in pix)  
  
def filter_im(im, filter_func, k):  
    return [[ filter_func(c,k) for c in row] for row in im ]  
  
images.visd(filter_im(im,luminosita,20))
```

21/11/2023

Operatore * per Pack e Unpack

Liste

Nel terminale, attraverso il semplice comando `*nome_della_lista`, rende una lista che contiene i singoli elementi dati dopo l'uguale. Nel caso seguente forniamo una stringa, quindi la lista conterrà le singole lettere.

```
In [2]: *fullname, = 'iacopo'
In [3]: fullname
Out[3]: ['i', 'a', 'c', 'o', 'p', 'o']

In [2]: *fullname, = 'iacopo', 'masi'
In [3]: fullname
Out[3]: ['iacopo', 'masi']
```

È possibile anche effettuare un Pack Parziale, o Partial Pack, inserendo più elementi a sinistra dell'uguale (=).

```
In [221]: *full_name, last = 'iacopo', 'lizardking', 'masi'
In [222]: full_name, last
Out[222]: (['iacopo', 'lizardking'], 'masi')
```

partial pack

Dizionari

La stessa cosa può essere tranquillamente fatta anche con i dizionari. È sufficiente mettere un doppio asterisco:

```
In [243]: def print_kwargs(**kwargs):
...:     for key, value in kwargs.items():
...:         print(f'{key} -> {value}')
```

`**kwargs` è un dizionario dove:

1. la chiave è il nome della variabile passato dal client
2. il valore è il contenuto della variabile passato dal client

Pack e Unpack nelle funzioni

Client

```
> func(*iterable)
```

call → unpack

Definizione

```
> def f(*args):
    pass
```

def → pack

Unire List e Dizionari (functional way)

```
In [243]: num_dict = {'a': 1, 'b': 2, 'c': 3}
...: num_dict_2 = {'d': 4, 'e': 5, 'f': 6}
...: print( {**num_dict, **num_dict_2} )
{'a': 1, 'b': 2, 'c': 3, 'd': 4, 'e': 5, 'f': 6}
```

dict ← {unpack, unpack}

{ ———— , ———— }

28/11/2023

Creare un rettangolo che comprende tutti gli altri, da top-left a bottom-right

Ricorsione

Una funzione si dice tale quando nella sua definizione la funzione chiama «se stessa» e vi sono definiti dei «casi base»

Numeri di Fibonacci

L'esempio classico per la Ricorsione è quello dei numeri di Fibonacci, in cui ad ogni numero restituisce la somma di tutti i numeri precedenti. Una prima ipotesi di ricorsione è la seguente:

```
class Fibo:
    n_calls = 0

    def fibonacci(n):
        Fibo.n_calls += 1

        if n < 2:
            return 1
        else:
            return Fibo.fibonacci(n-1) + Fibo.fibonacci(n-2)
```

Tuttavia richiamando la funzione due volte andremo a creare una sorta di “struttura ad albero” che avrà 2^{n-1} rami. Il che significa un sacco di ricorsioni.

Tuttavia questo algoritmo può essere migliorato. Per evitare di entrare in un ramo che abbiamo già calcolato possiamo memorizzare il valore e successivamente attraverso caching tirarlo fuori. Parlando di caching pensiamo immediatamente ai dizionari. Possiamo quindi creare un dizionario vuoto ed aggiungerci all'interno i valori corrispondenti ad ogni numero che abbiamo visitato e poi, alla ricorsione successiva, controllare se quel numero è già in memoria (cioè nel dizionario):

```
def fibonacci_memo(n):
    """
    Fibonacci with recursion + memoization
    """
    memory = { 0 : 1, 1 : 1}
    if n in memory:
        return memory[n]
    else:
        rez = fibonacci_memo(n-1) + fibonacci_memo(n-2)
        memory[n] = rez
        return rez
```

Per finire, ogni esercizio ricorsivo può essere risolto anche in maniera iterativa:

```
def fibonacci_iter(n):
    """
    Iterative fibonacci
    """
    f = [1,1] + [None]*(n-1)
    for k in range(2,n+1):
        f[k] = f[k-1] + f[k-2]
    return f, f[n]
```

30/11/2023

Consigli

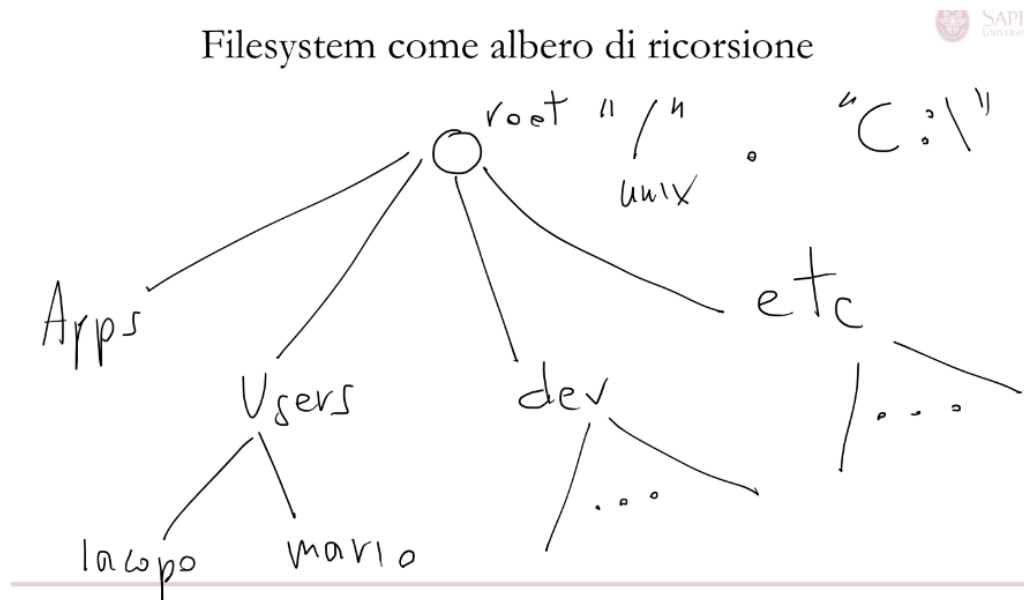
Per gli esercizi sulla Ricorsione c'è una sorta di Ricetta Generica che può tornarci utile in tutti gli esercizi:

- 1) [Riduzione] È possibile rimpicciolire la dimensione del problema. (richiamare il caso n-1)
- 2) [Case base] Esiste almeno un problema con una soluzione elementare. (Solitamente lo rappresentiamo con un if all'inizio della funzione che 'cattura' il caso base)
- 3) [Convergenza] È sempre possibile, applicando lo step di riduzione, di arrivare ad un caso base.
- 4) [Conquer] Unisci le soluzioni delle riduzioni per risolvere il problema principale.

05/12/2023

Navigare il FileSystem Ricorsivamente (99% all'esame)

Nel FileSystem i nodi dell'albero sono le cartelle, mentre le foglie sono i file veri e propri. Supponiamo ora di voler trovare tutti i file con una determinata estensione.



Trovare i file in base all'estensione

Per controllare se sono directory o files possiamo usare il comando `os.path.isdir()` oppure `os.path.isfile()`

Con `os.listdir()` otteniamo una lista di stringhe che descrivono il percorso da fare per raggiungere ciascun file 'gerarchicamente' sotto di noi. In pratica otteniamo il percorso relativo di tutti i file, dal punto in cui ci troviamo in poi.

Per ottenere il percorso assoluto dobbiamo concatenare il path in cui ci troviamo con il percorso relativo dei files sotto di noi.

- **path.exists**
verifica l'esistenza di un file
- **path.isdir**
verifica l'esistenza di una cartella
- **path.getsize**
ritorna le dimensioni di un file
- **path.dirname**
estrae la directory dal pathname
- **path.basename**
estrae il nome del file dal pathname
- **path.join**
unisce il dirname al basename
- **path.split**
separa il path dal nome del file nel pathname
- **path.splitext**
separa l'estensione del file dal nome

```
def find_file_with_ext(folder, ext):  
    rez = []  
    for filename in os.listdir(folder):  
        #ogni percorso sotto di noi è un altro ramo del FileSystem  
        #quindi itero finchè ho controllato tutti i file  
        full_path=folder + '/' + filename  
        #creo il path assoluto  
        if os.path.isfile(full_path) and full_path.endswith(ext):  
            #se il path è un file e termina con l'estensione prevista  
            rez.append(full_path)  
        else:  
            #se non è un file sicuramente è una sottocartella e  
            #richiamo la funzione da capo  
            List_files = find_file_with_ext(folder, ext)  
            #calcolo la funzione e la inserisco in una variabile 'appoggio'  
            rez.append(List_files)  
            #inserisco il risultato della funzione nella lista di ritorno  
    return rez
```

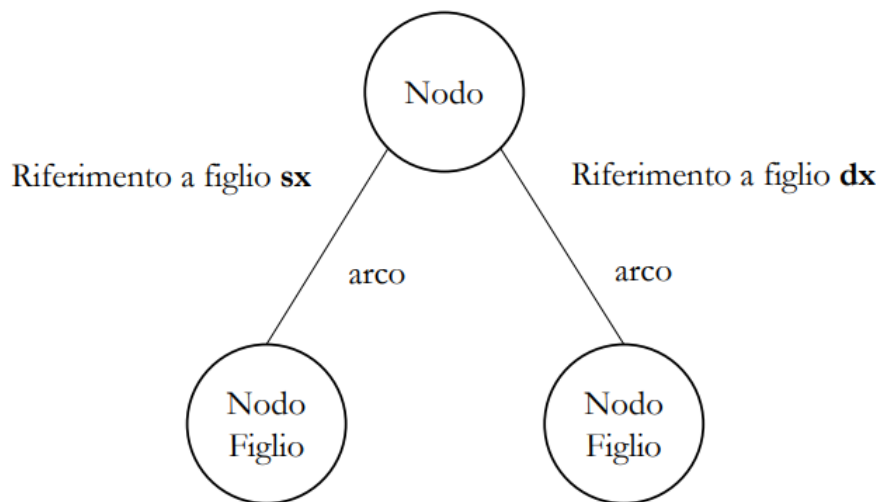
Trovare i file in base al Peso in Byte

Se invece volessimo returnare un dizionario che ad ogni path restituisce la grandezza in byte possiamo usare lo stesso procedimento per la ricerca e successivamente calcolare il peso.

Per farlo possiamo usare `pippo.st_size`

```
def find_file_with_size(folder, ext):
    rez = {}
    for fname in os.listdir(folder):
        # mi rende i file e dir in folder
        # mi riprocuro il percorso assoluto
        full_path = f"{folder}/{fname}"
        if os.path.isfile(full_path):
            # se siamo nel caso del file controllo estensione
            if full_path.endswith(ext):
                # ok lo aggiungo alla lista la dimensione
                # in byte tramite modulo os.stat
                rez[full_path] = os.stat(full_path).st_size
                # result[full_path] = os.stat(full_path).st_size
            else:
                # ottengo il dizionario per le sotto cartelle
                D_files = find_file_with_size(full_path, ext)
                # unisco i due dizionari
                # si puo' anche fare con update invece che
                # con unpack
                rez.update(D_files)
                #rez = {**rez, **D_files}
    # torno per eventuali chiamate sopra di me
    return rez
```

Alberi Binari

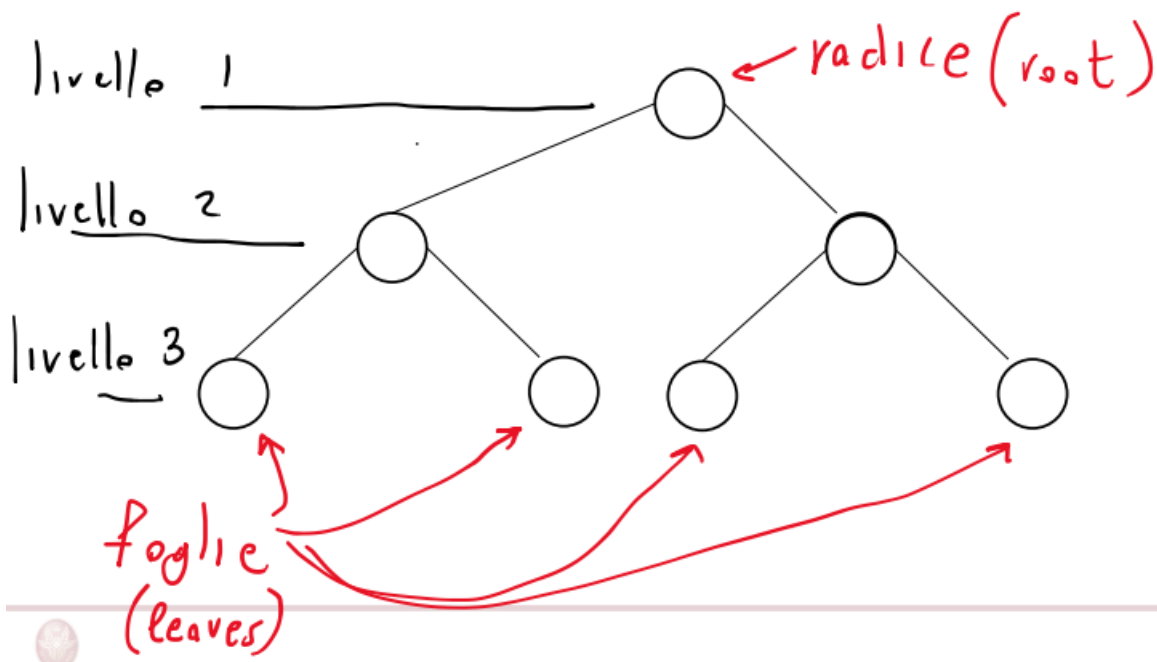


Per poter lavorare con un albero binario, a livello di codice, avremo bisogno di creare una nuova classe tenendo conto del fatto che:

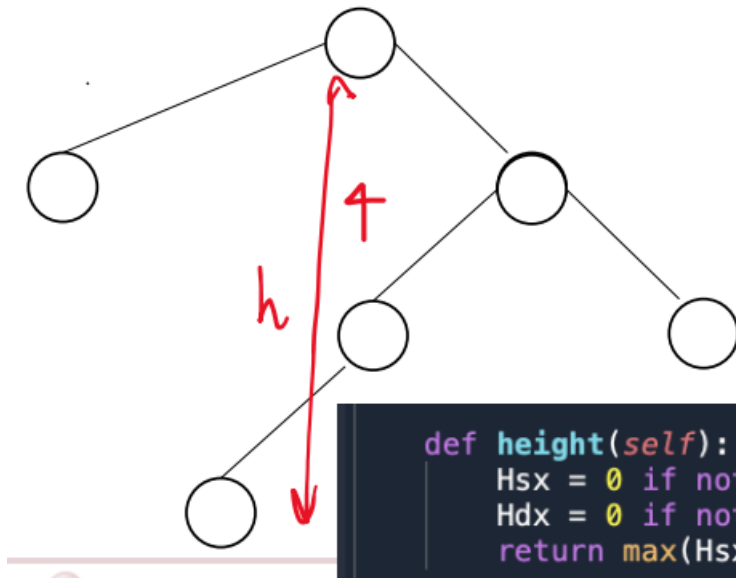
- Ad ogni nodo associamo un valore
- Ogni nodo può avere un a sua volta dei "nodi figlio" sia a destra che a sinistra

```
class BinaryNode:  
    def __init__(self, value, sx=None, dx=None):  
        self.value = value  
        self.sx = sx  
        self.dx = dx
```

Esempio

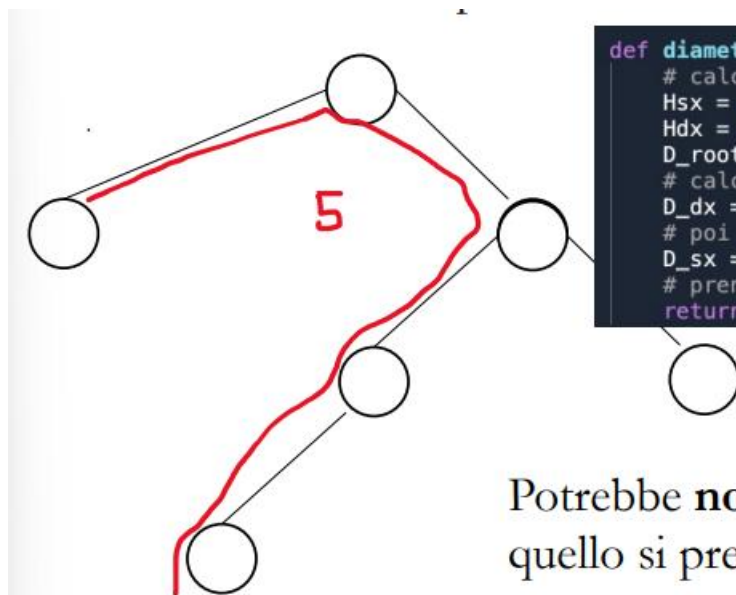


Livello e Altezza



```
def height(self):  
    Hsx = 0 if not self.sx else self.sx.height()  
    Hdx = 0 if not self.dx else self.dx.height()  
    return max(Hsx, Hdx) + 1
```

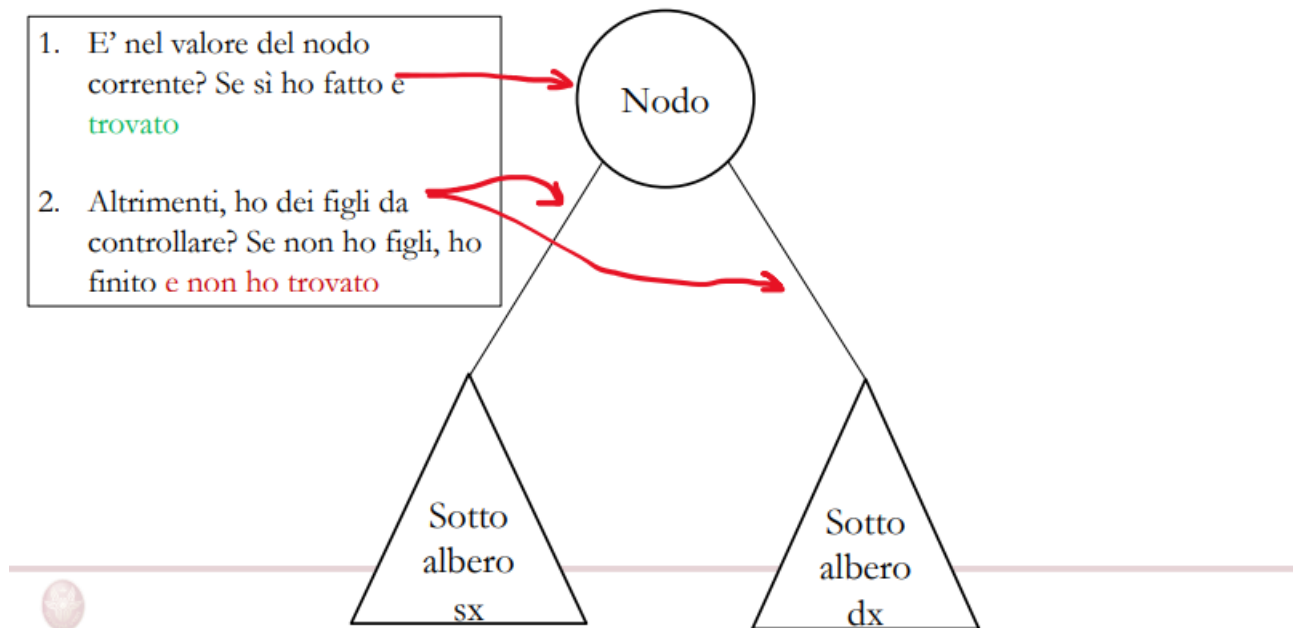
Calcolare il Percorso Massimo (anche detto Diametro)



```
def diameter(self):  
    # calcolo percorso massimo su radice  
    Hsx = 0 if not self.sx else self.sx.height()  
    Hdx = 0 if not self.dx else self.dx.height()  
    D_root = Hsx + Hdx + 1  
    # calcolo percorso max. sul destro  
    D_dx = 0 if not self.dx else self.dx.diameter()  
    # poi sul sinistro  
    D_sx = 0 if not self.sx else self.sx.diameter()  
    # prendo il massimo  
    return max(D_root, D_dx, D_sx)
```

Potrebbe **non** passare dalla radice (per quello si prende il max fra i tre)

Ricerca in un Albero (Ricorsivamente)



Caso Base

Immaginiamo di avere una qualsiasi struttura ad albero. In quale momento della ricerca possiamo stabilire che non ci sono altri elementi da cercare? Cioè che siamo arrivati al Caso Base e possiamo restituire i valori?

Le opzioni sono 2:

- Trovo la query all'interno di un nodo
- NON trovo la query in nessun nodo e la foglia sotto di me non ha altri figli da controllare (né a dx né a sx)

```
def find(self, query):  
    if self.value == query:  
        return True
```



```
def find(self, query):  
    if self.value == query:  
        return True  
    elif not self.sx and not self.dx:  
        return False
```

Passo Ricorsivo

Dopo aver individuato il caso base dobbiamo solamente implementare la ricorsione al fine di scorrere tutto l'albero. E lo facciamo nel caso in cui non abbiamo ancora trovato il valore e sotto di noi ci sono sottoalberi (dx/sx) su cui possiamo applicare la ricerca:

```
def find(self, value):  
    if self.value == value:  
        return True  
    elif not self.sx and not self.dx:  
        return False  
    elif self.sx: #esiste sottoalbero SX  
        if self.sx.find(value):  
            return True  
    elif self.dx: #esiste sottoalbero DX  
        return self.dx.find(value)  
    else:  
        return False
```


Ricerca in Pre-Order

Non cambia molto dalla ricerca normale, semplicemente andiamo ad unire alcuni passaggi così da essere più rapidi. In particolare i casi base restano gli stessi ma scritti in modo diverso. Di seguito il codice:

```
def find_pre_order(self, query):  
    if self.value == query:  
        return True  
    elif self.sx and self.sx.find_pre_order(query):  
        return True  
    elif self.dx and self.dx.find_pre_order(query):  
        return True  
    else:  
        return False
```

1. **Se il valore del nodo in cui mi trovo è uguale a quello che sto cercando** ho finito e torno True (perché l'ho trovato!!). Se non è nel nodo controllo:
2. **Se esiste sottoalbero** → **applico la funzione ricorsiva** (che torna un valore True o False). Se esiste e trovo qualcosa avremo True and True quindi possiamo dire che abbiamo trovato la query. Se il sottoalbero sinistro NON esiste passo immediatamente al sottoalbero destro:
3. **Se esiste sottoalbero destro** → **applico la funzione ricorsiva** (esattamente come prima) ed avremo True se nel sottoalbero destro troviamo la query che stiamo cercando.
4. **Se nessuno dei casi precedenti è True** → **return False** perché in nessun nodo dell'albero abbiamo trovato la query.

Ricerca In Order

Ricerca in Post Order

12/12/2023 (Ero assente)

Alberi da Gioco

Immaginiamo il gioco tris